Architecture 1001:
x86-64 Assembly
Mystery Lystery 1 Solved!

Xeno Kovah – 2021
xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"

Mystery Listery

- Why do the GCC/Clang HelloWorlds have balanced Push/Pop instructions but Visual Studio doesn't?
- What's up with the sub/add 0x28 in main()?
- Why is VS over-allocating space for a single local variable?

SOLVED!

SOLVED!

Mystery Listery

- Why do the GCC/Clang HelloWorlds have balanced Push/Pop instructions but Visual Studio doesn't?
- What's up with the sub/add 0x28 in main()?
- Why is VS over-allocating space for a single local variable?

SOLVED!

SOLVED!

As seen in Hello.c! (on macOS)

```
_main:
0000000100003f50        pushq   %rbp
0000000100003f51        movq    %rsp, %rbp
0000000100003f54        subq    $0x10, %rsp
0000000100003f58        movl    $0x0, -0x4(%rbp)
0000000100003f5f        leaq    0x38(%rip), %rdi
0000000100003f66        movb    $0x0, %al
0000000100003f68        callq   0x100003f7e ##_printf
0000000100003f6d        movl    $0x1234, %ecx
0000000100003f72        movl    %eax, -0x8(%rbp)
0000000100003f75        movl    %ecx, %eax
0000000100003f77        addq    $0x10, %rsp
0000000100003f7b        popq    %rbp
0000000100003f7c        retq
```

macOS 11.2, Apple clang version 12.0.0 (clang-1200.0.32.29)
Disassembled from command line with “otool -tV”

As seen in Hello.c! (on Linux)

```
0000000000001149 <main>:
    1149:  endbr64
    114d:  push   %rbp
    114e:  mov    %rsp,%rbp
    1151:  lea    0xeac(%rip),%rdi
    1158:  callq  1050 <puts@plt>
    115d:  mov    $0x1234,%eax
    1162:  pop    %rbp
    1163:  retq
    1164:  nopw   %cs:0x0(%rax,%rax,1)
    116e:  xchg   %ax,%ax
```

Ubuntu 20.04, GCC 9.3.0
Disassembled with “objdump -d”

Parameter Conventions: Microsoft “x64” ABI

- First 4 parameters (from left to right) are put into RCX, RDX, R8, R9 respectively
- Remaining parameters “pushed” onto the stack so that the left-most parameter is at the lowest address

Parameter Conventions: Microsoft “x64” ABI

- First 4 parameters (from left to right) are put into RCX, RDX, R8, R9 respectively
- Remaining parameters “pushed” onto the stack so that the left-most parameter is at the lowest address
- *Generally doesn't use of frame pointers for most code*

64-bit Maximum Stack Diagram - VS

stack bottom

stack top

64-bit Maximum Stack Diagram - VS

stack bottom

“return address” from main() to whoever called main()

stack top

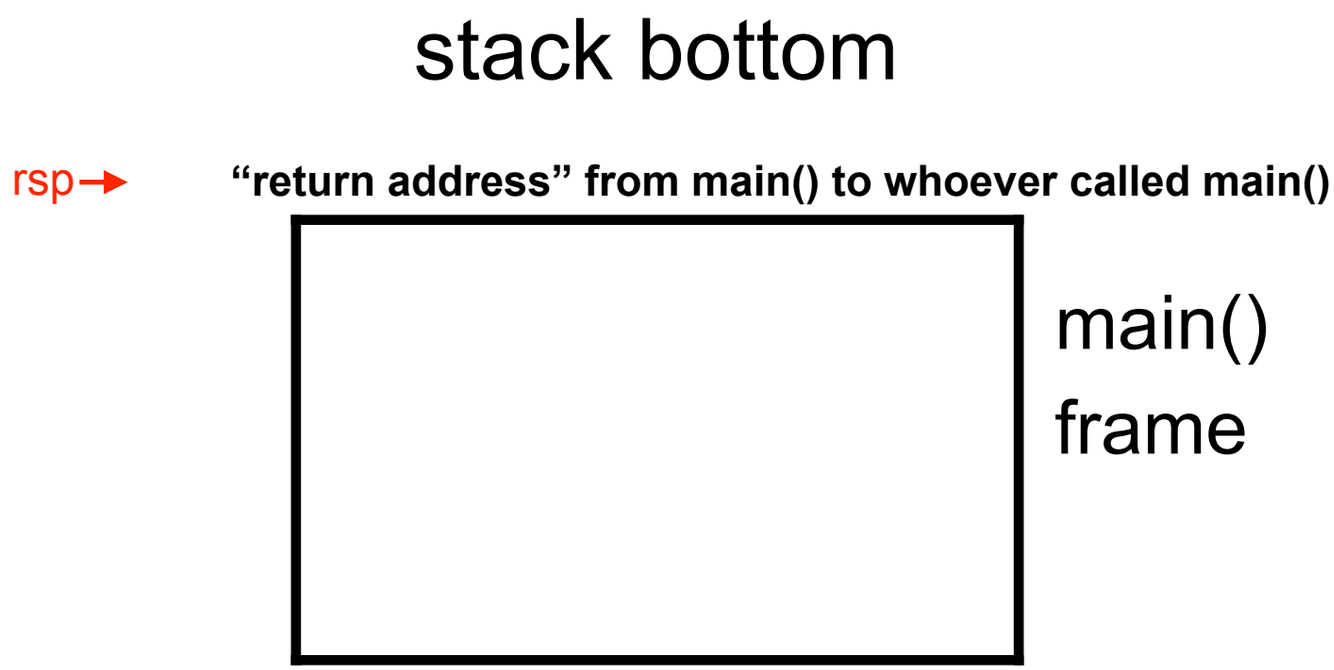
64-bit Maximum Stack Diagram - VS

stack bottom

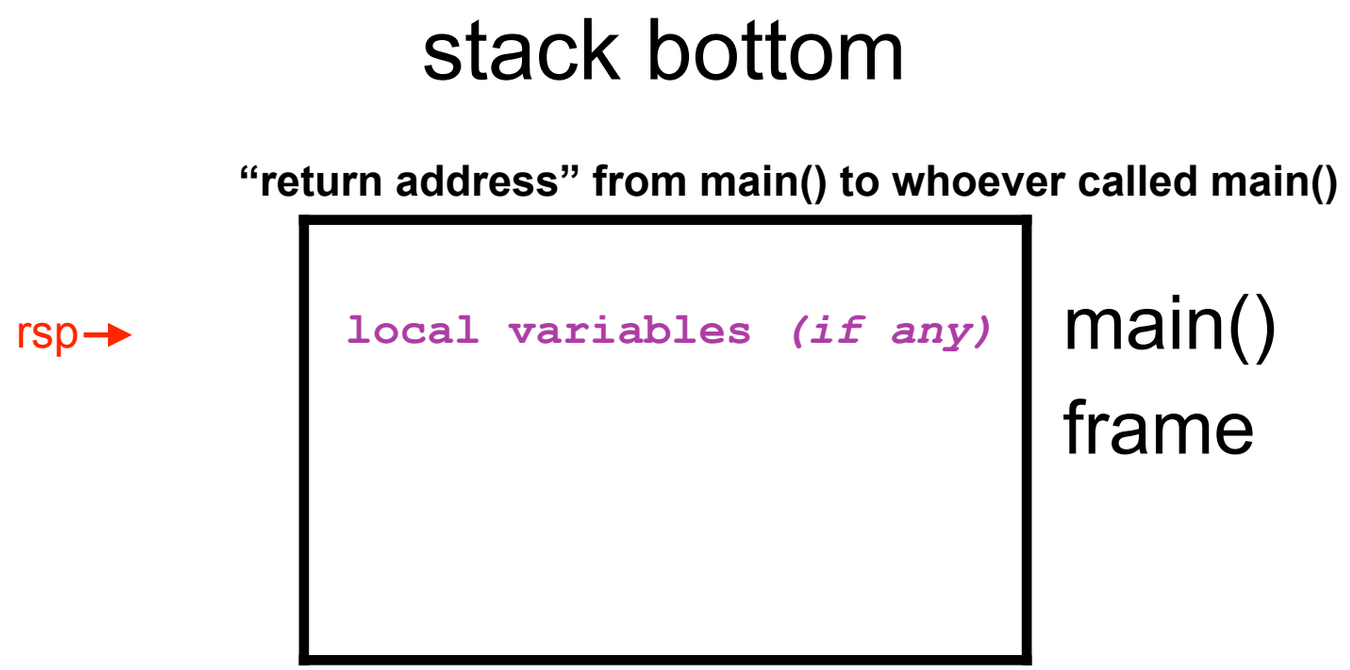
rsp → **“return address” from main() to whoever called main()**

stack top

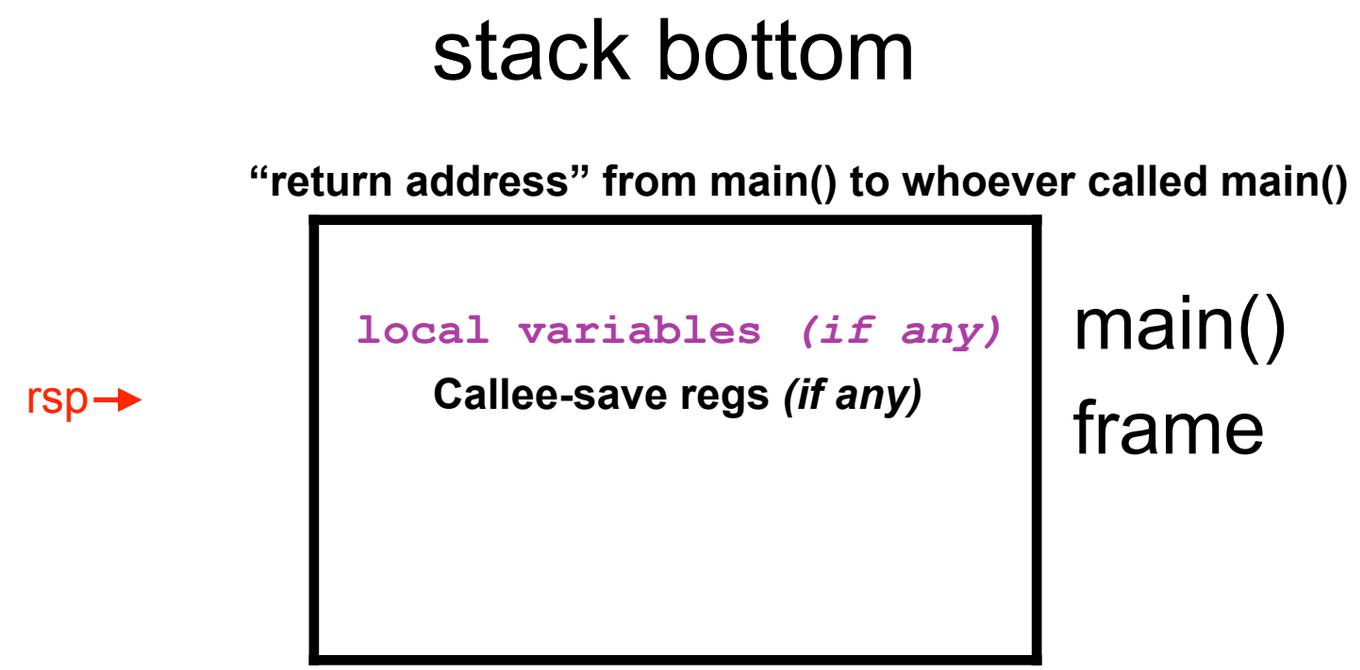
64-bit Maximum Stack Diagram - VS



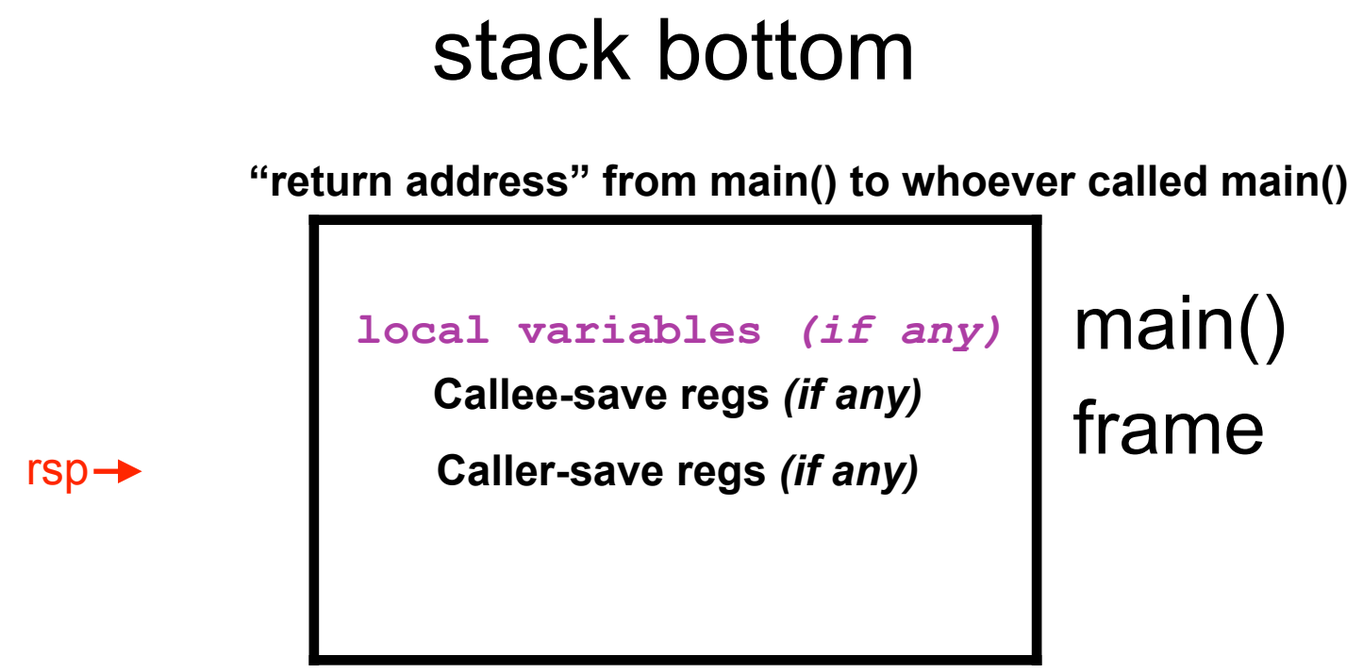
64-bit Maximum Stack Diagram - VS



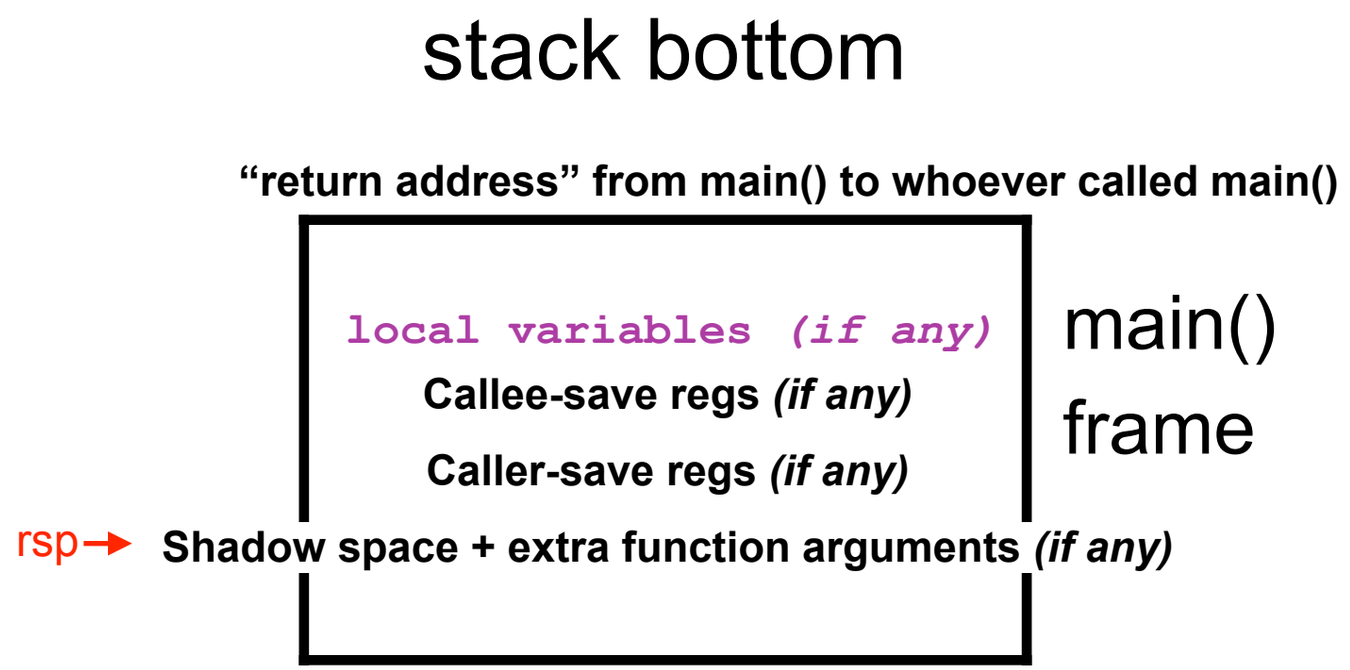
64-bit Maximum Stack Diagram - VS



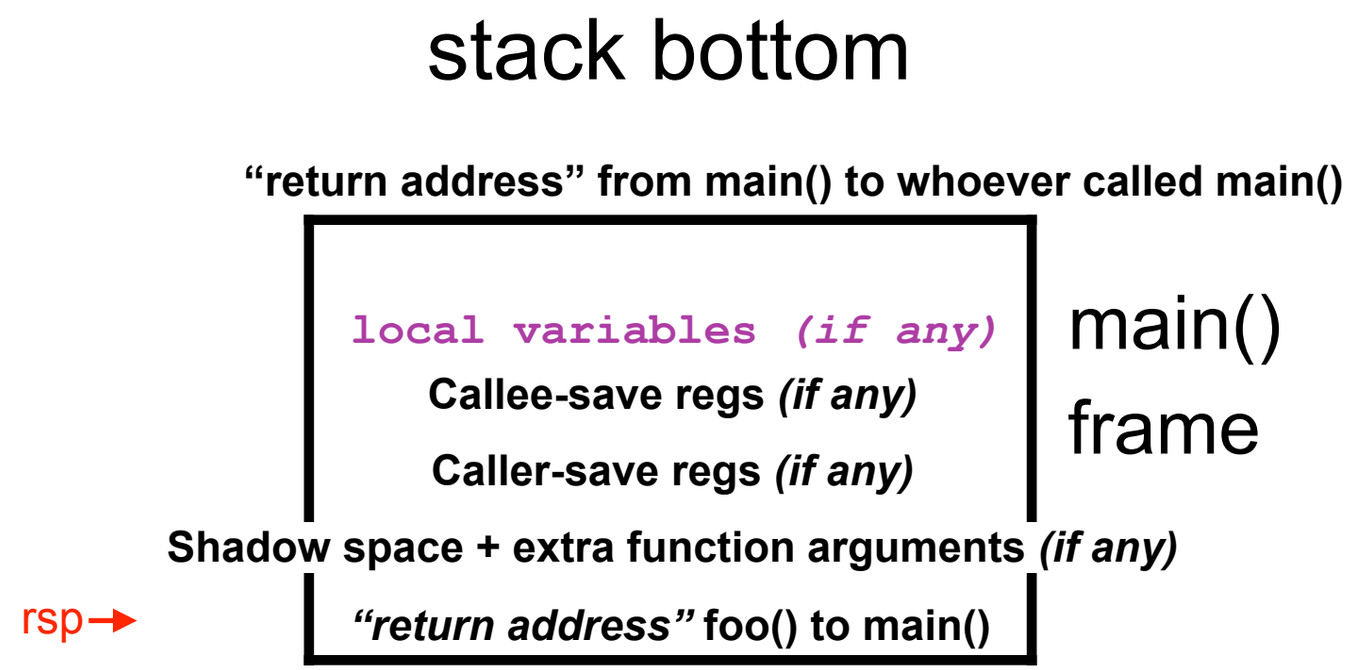
64-bit Maximum Stack Diagram - VS



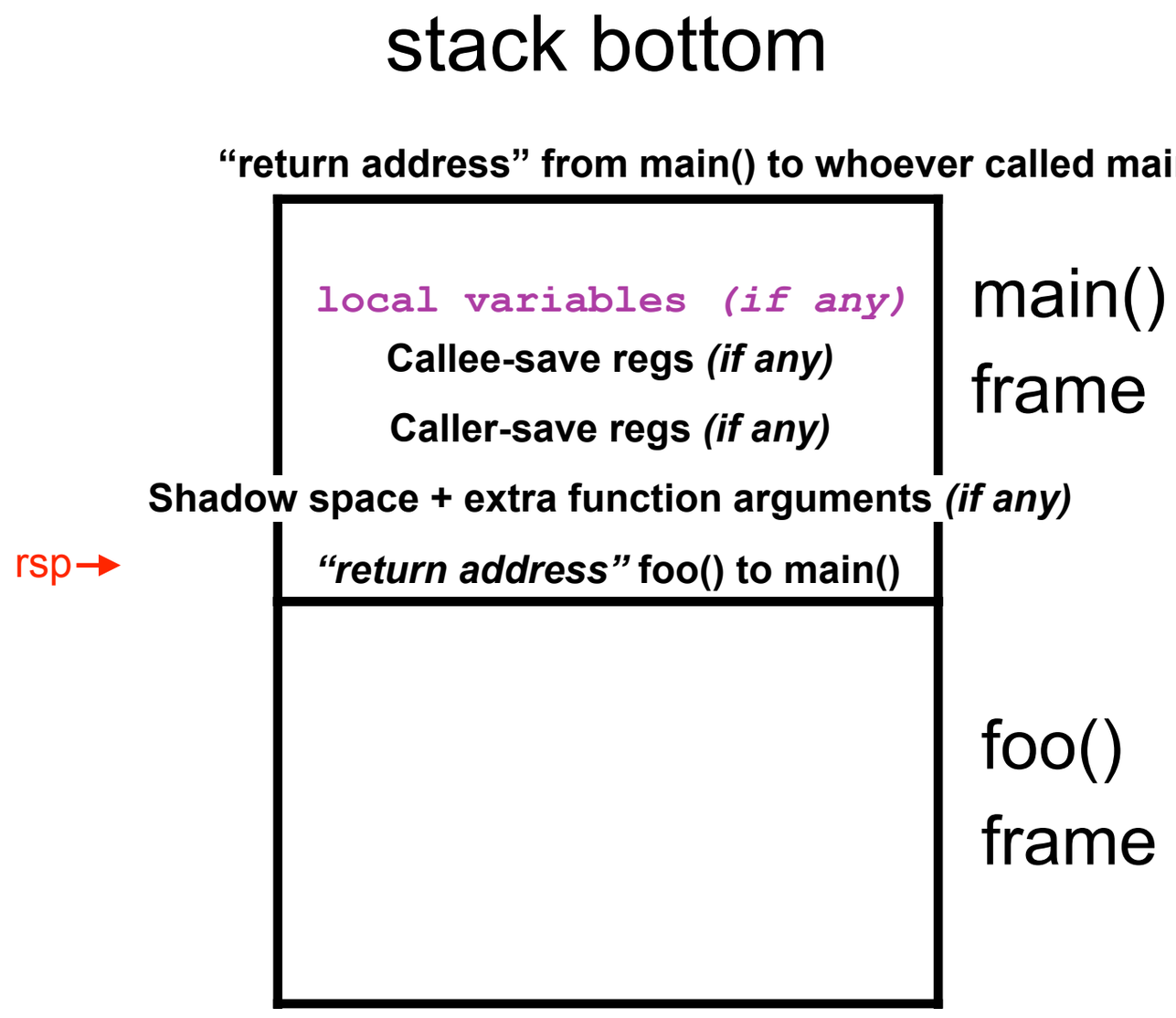
64-bit Maximum Stack Diagram - VS



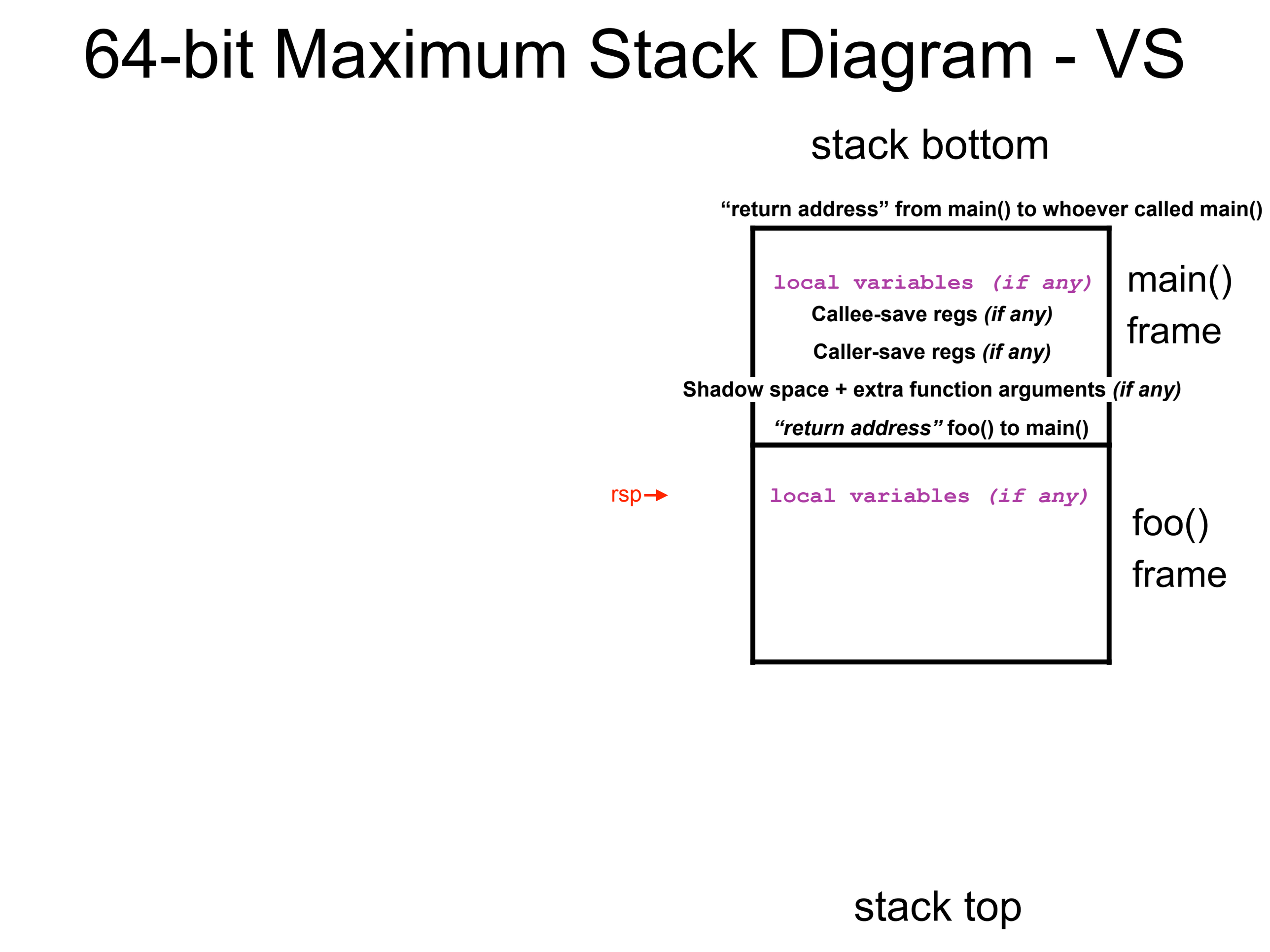
64-bit Maximum Stack Diagram - VS



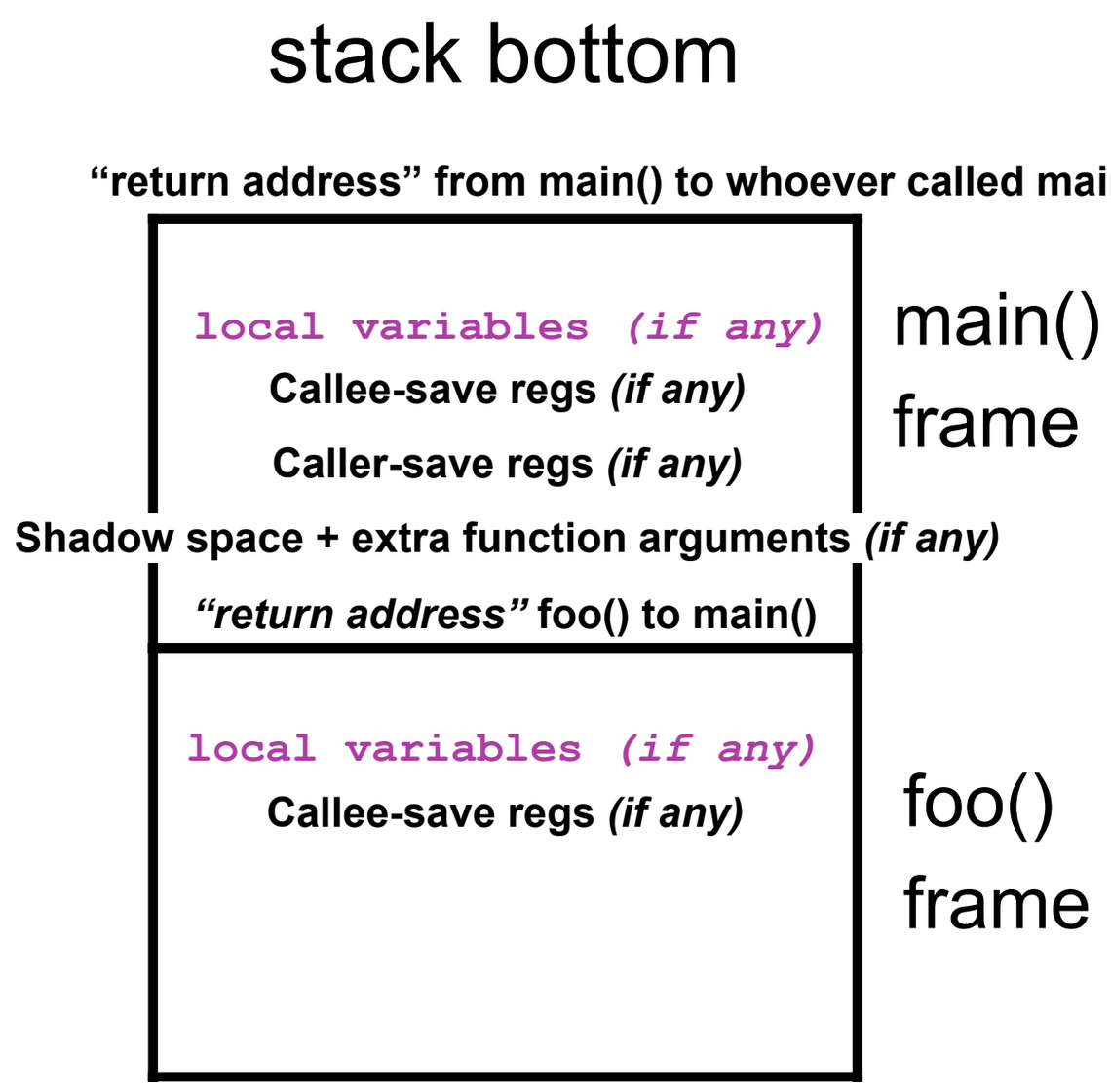
64-bit Maximum Stack Diagram - VS



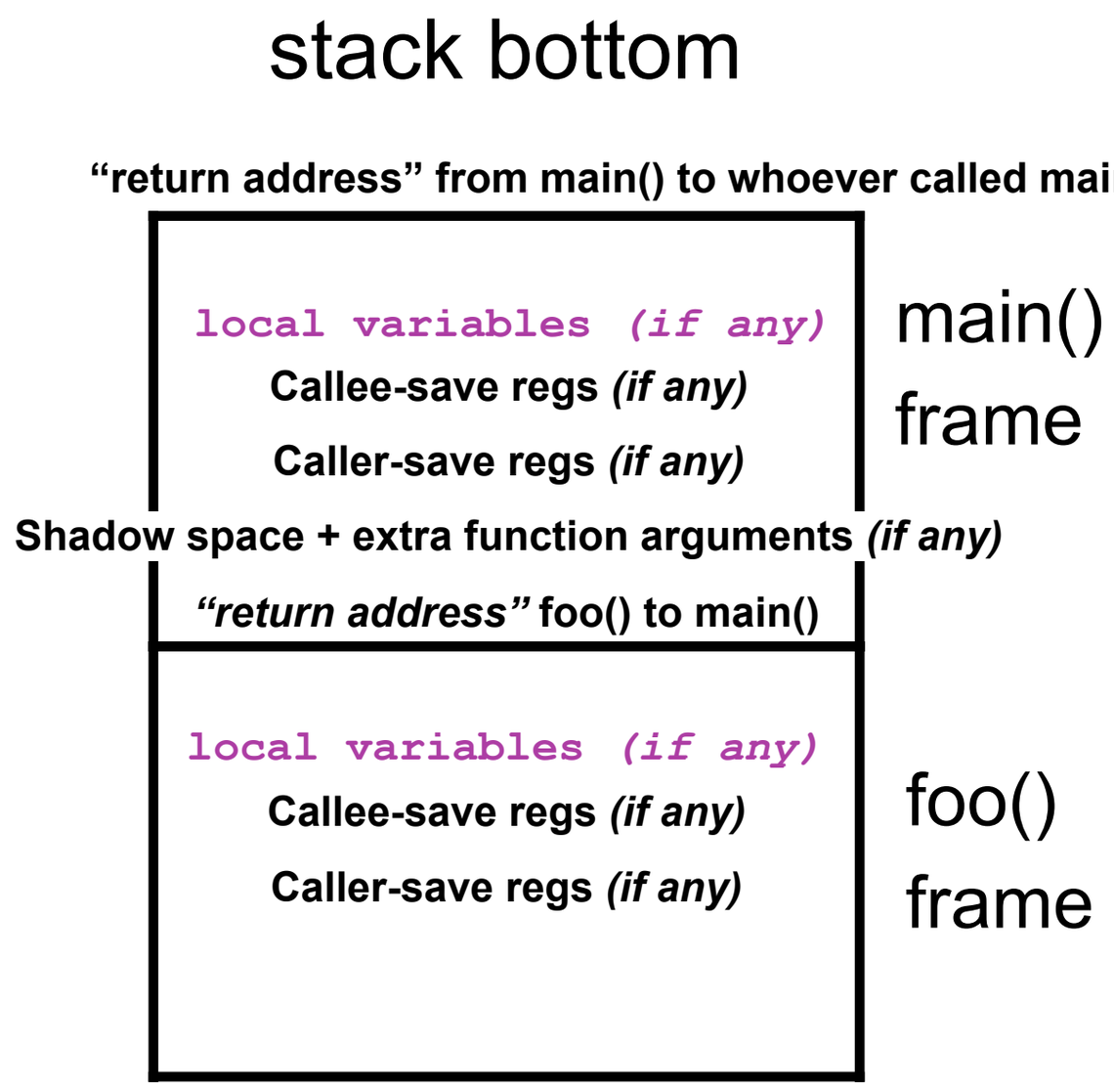
64-bit Maximum Stack Diagram - VS



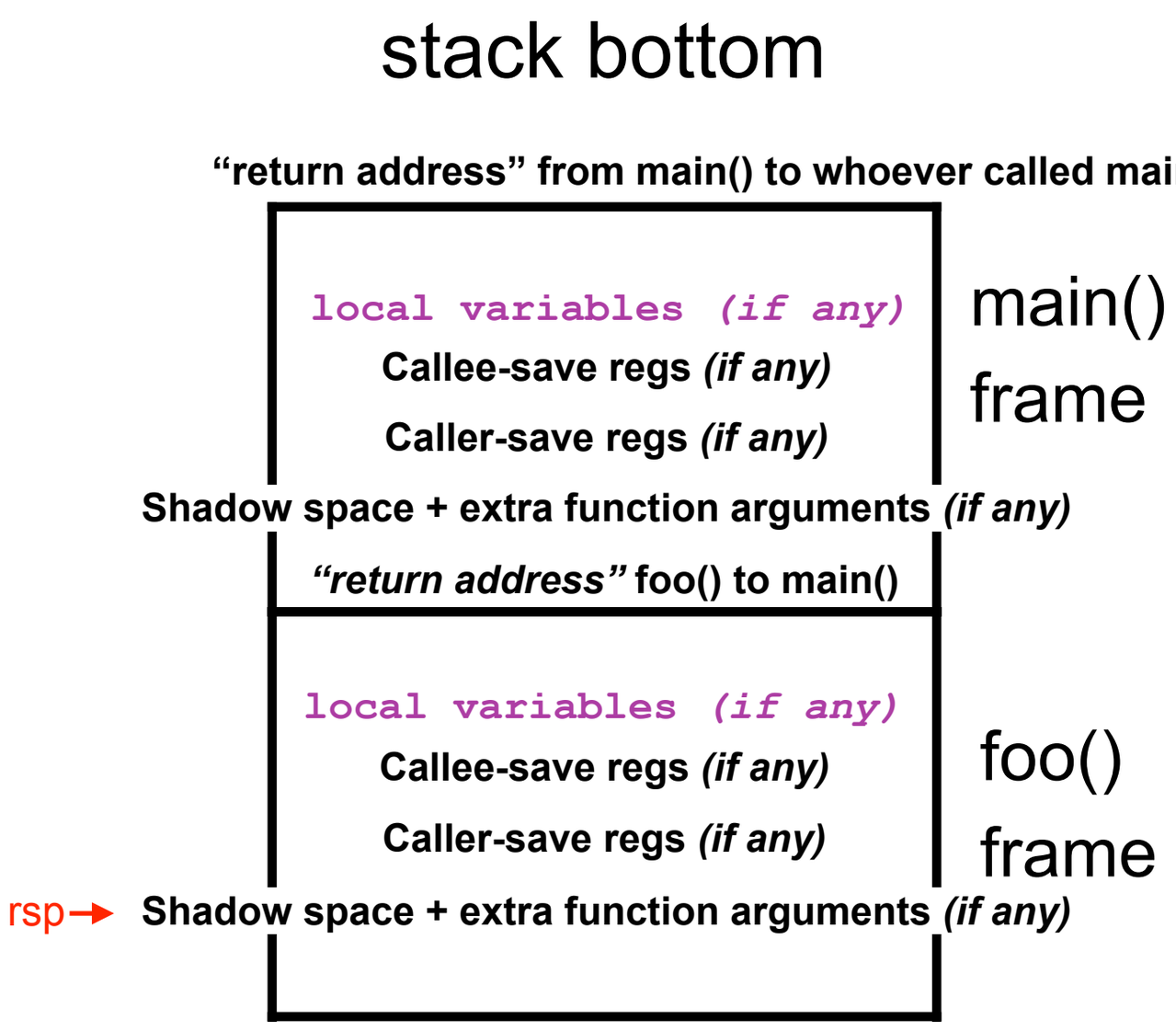
64-bit Maximum Stack Diagram - VS



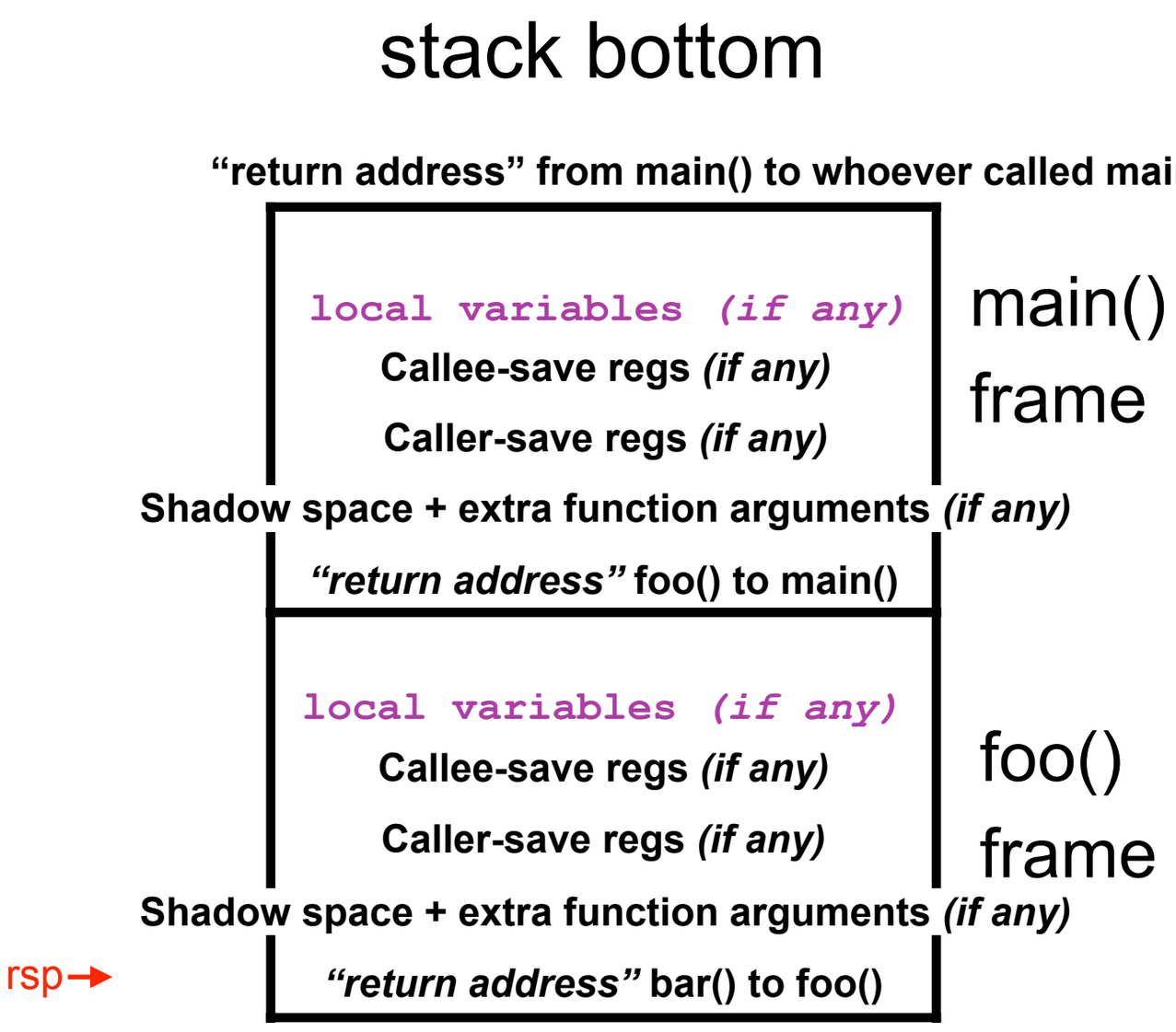
64-bit Maximum Stack Diagram - VS



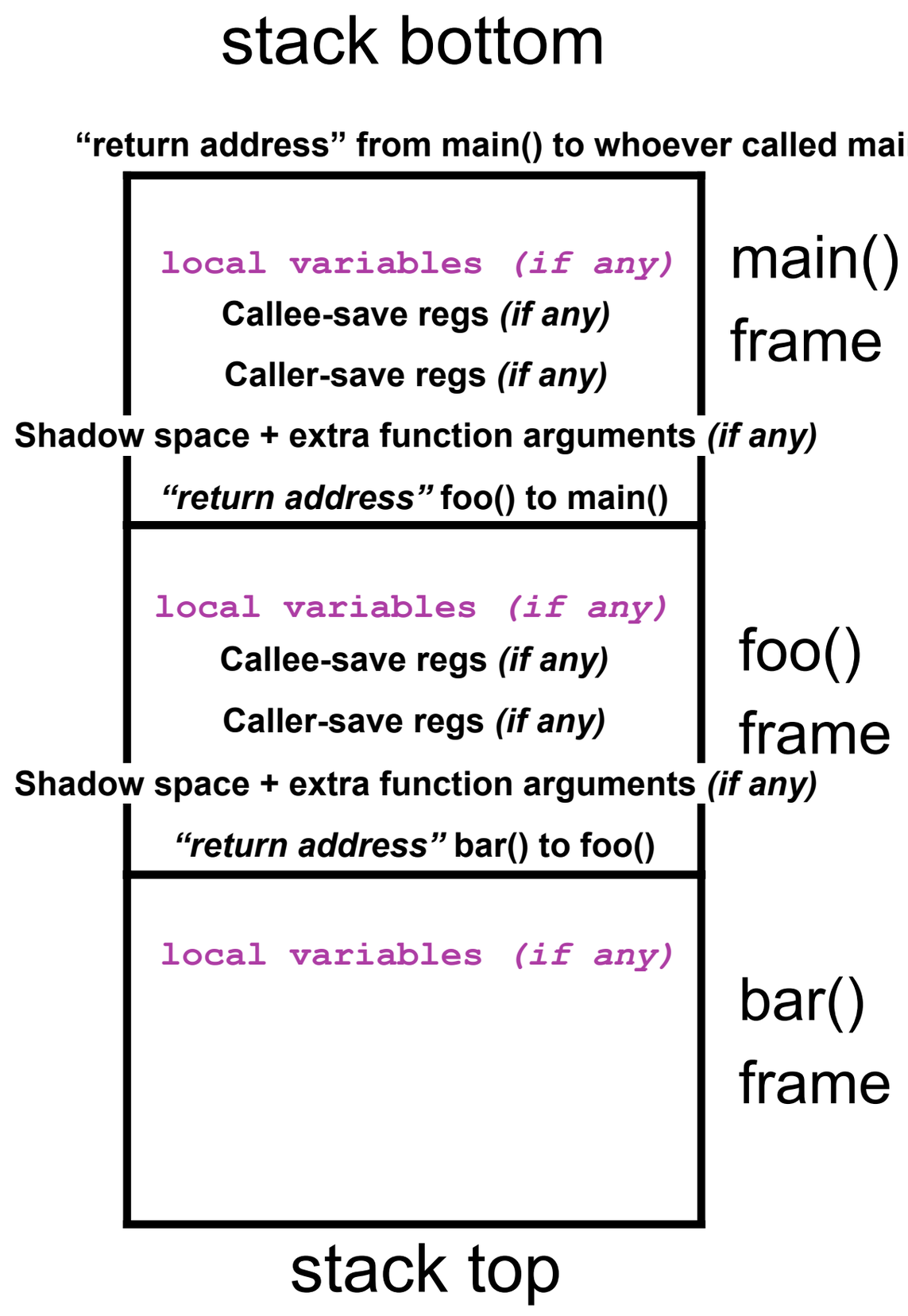
64-bit Maximum Stack Diagram - VS



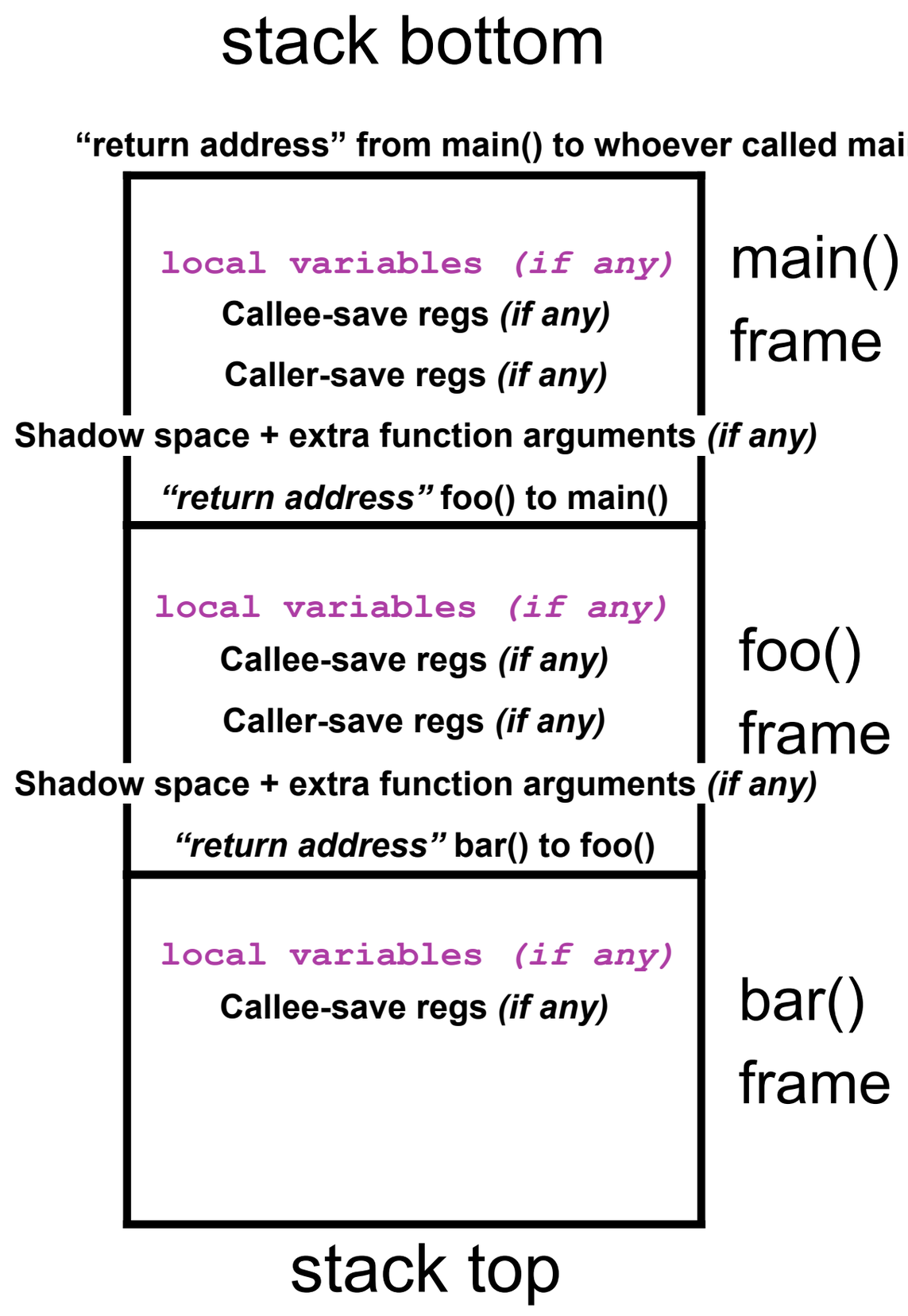
64-bit Maximum Stack Diagram - VS

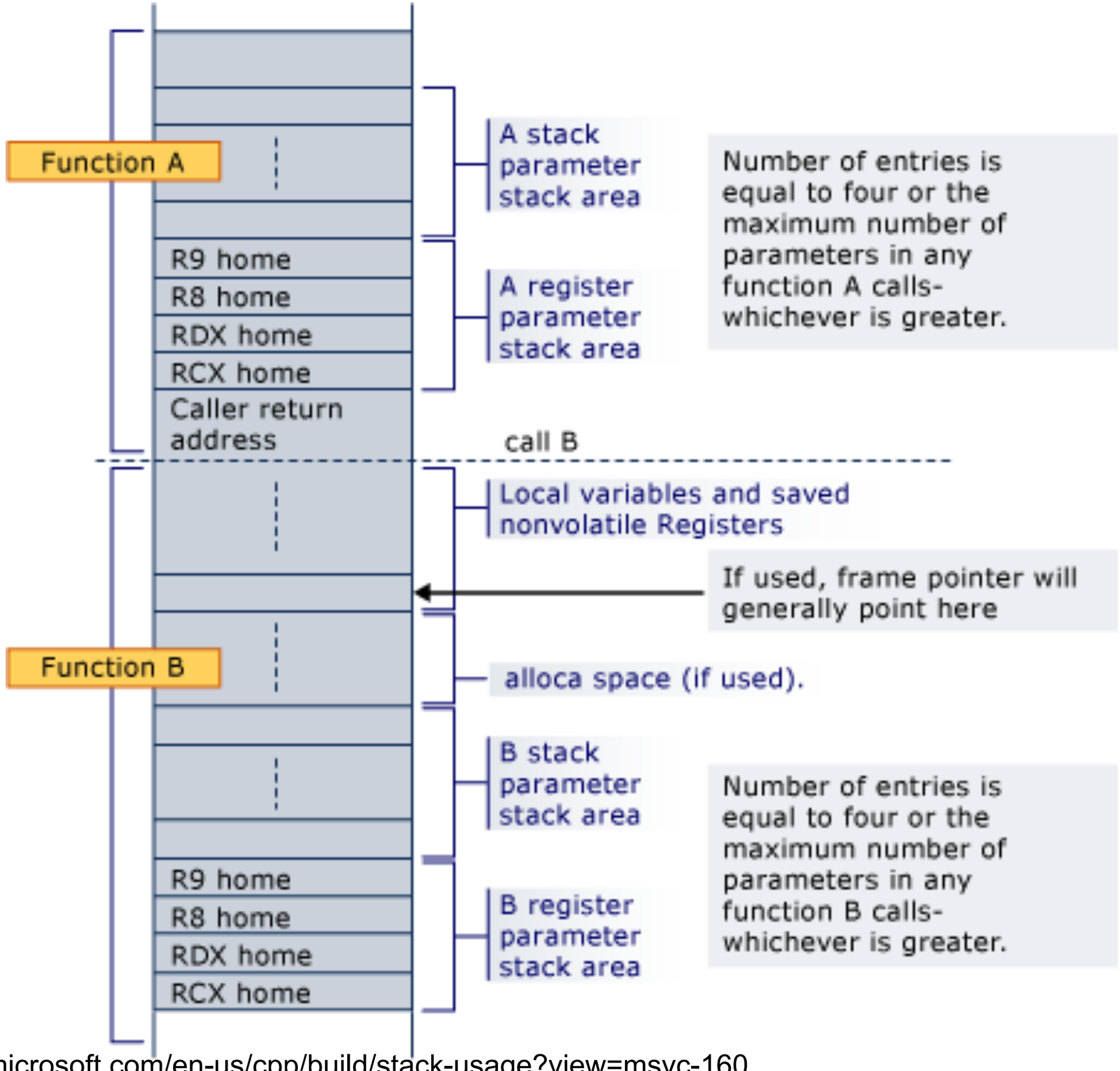


64-bit Maximum Stack Diagram - VS

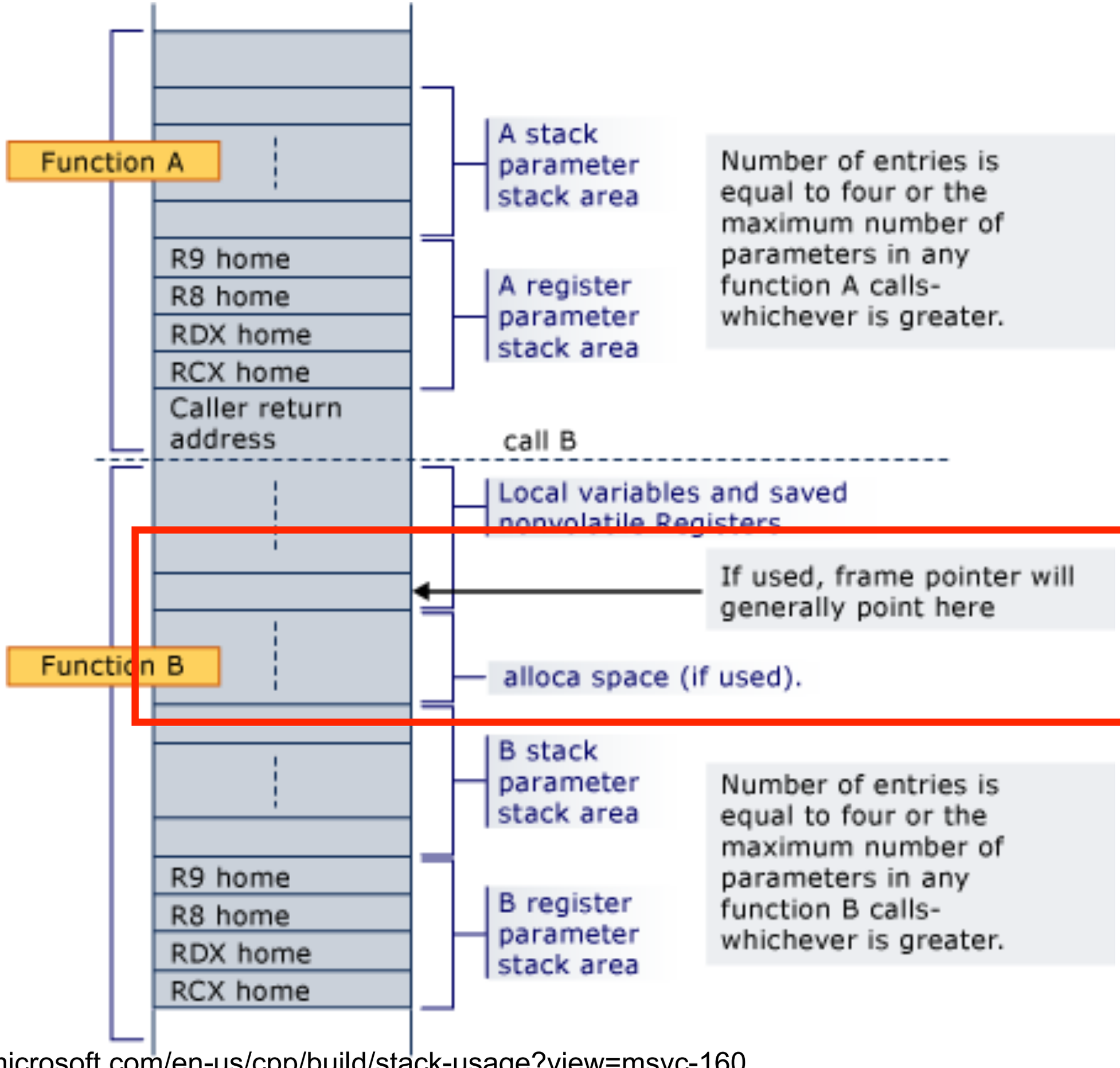


64-bit Maximum Stack Diagram - VS



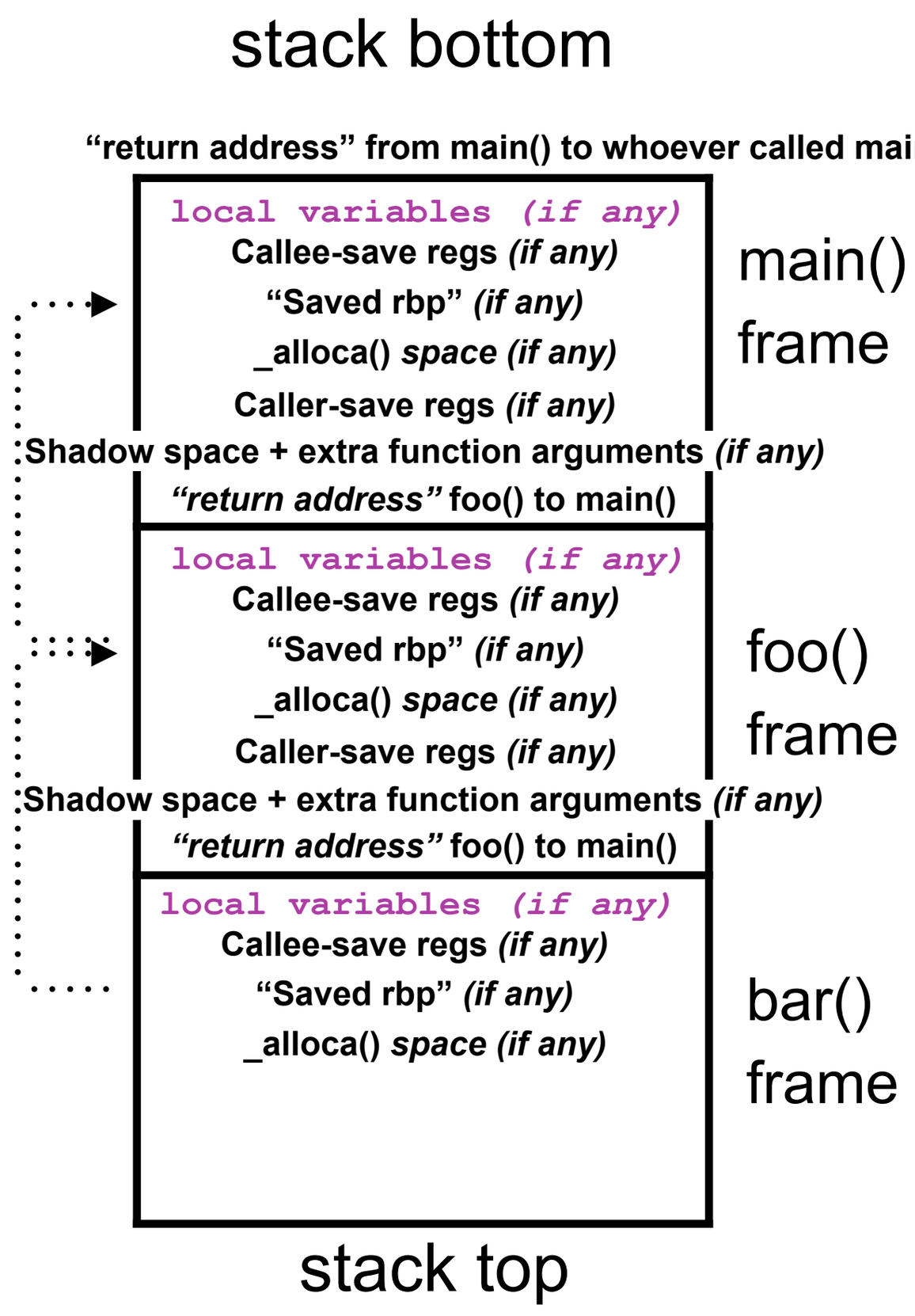


https://docs.microsoft.com/en-us/cpp/build/stack-usage?view=msvc-160



https://docs.microsoft.com/en-us/cpp/build/stack-usage?view=msvc-160

64-bit Maximum Stack Diagram - VS++



When *Does* VS Use a Frame Pointer?

- From [1] it definitely does if _alloca() is used
  - "_alloca is required to be 16-byte aligned and additionally required to use a frame pointer."
  - "If space is dynamically allocated (alloca) in a function, then a nonvolatile register must be used as a frame pointer to mark the base of the fixed part of the stack and that register must be saved and initialized in the prolog."
    - Note it said "a nonvolatile register", it didn't say it had to be rbp. Elsewhere [2] they have an example using r13 instead of rbp
- Not clear if there are any other situations

[1] https://docs.microsoft.com/en-us/cpp/build/stack-usage?view=msvc-160

[2] https://docs.microsoft.com/en-us/cpp/build/prolog-and-epilog?view=msvc-160

Parameter Conventions: System V “x86-64” ABI (GCC et al.)

- First 6 parameters (from left to right) are put into RDI, RSI, RDX, RCX, R8, R9 respectively
- Remaining parameters “pushed” onto the stack so that the left-most parameter is at the lowest address

Parameter Conventions: System V “x86-64” ABI (GCC et al.)

- First 6 parameters (from left to right) are put into RDI, RSI, RDX, RCX, R8, R9 respectively
- Remaining parameters “pushed” onto the stack so that the left-most parameter is at the lowest address
- *Still use frame pointers in the 32-bit style, unless directed not to by compile options*

64-bit Maximum Stack Diagram - SysV

stack bottom

stack top

64-bit Maximum Stack Diagram - SysV

rbp→

stack bottom

stack top

64-bit Maximum Stack Diagram - SysV

rbp →

stack bottom

“return address” from main() to whoever called main()

stack top

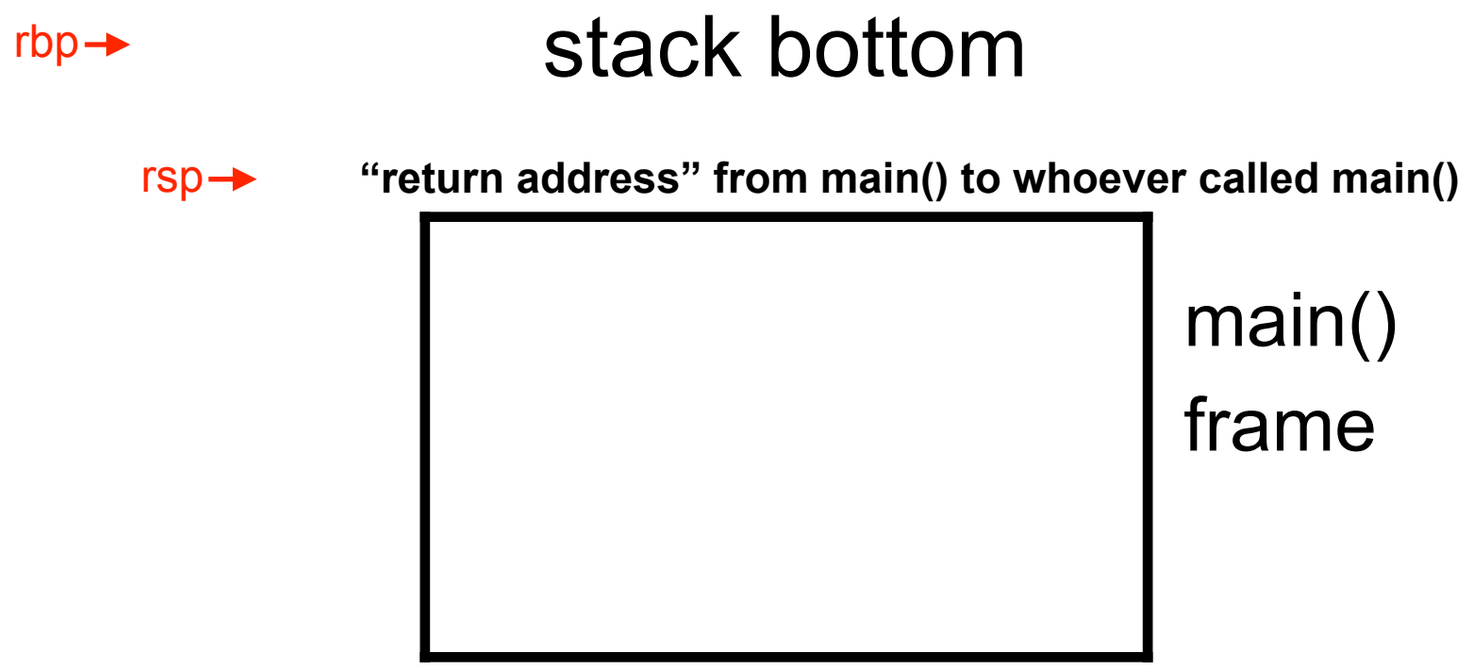
64-bit Maximum Stack Diagram - SysV

rbp → stack bottom

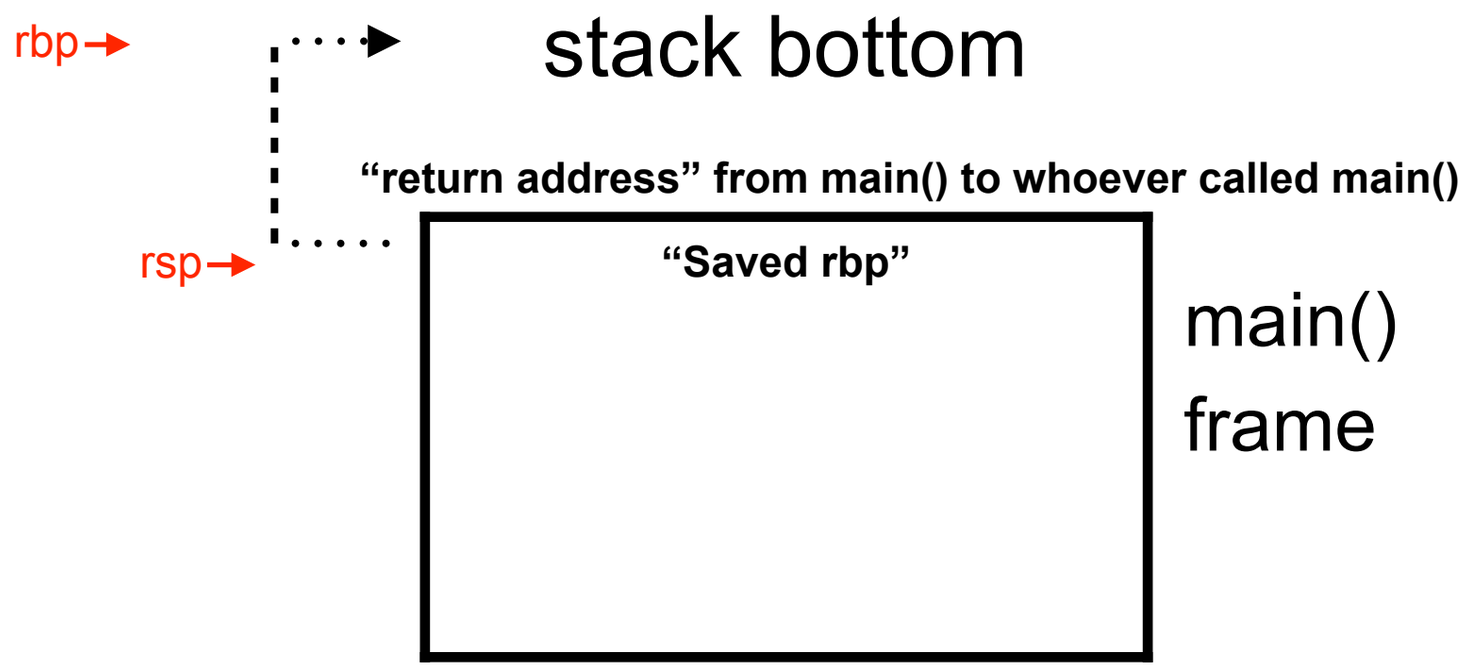
rsp → **“return address” from main() to whoever called main()**

stack top

64-bit Maximum Stack Diagram - SysV

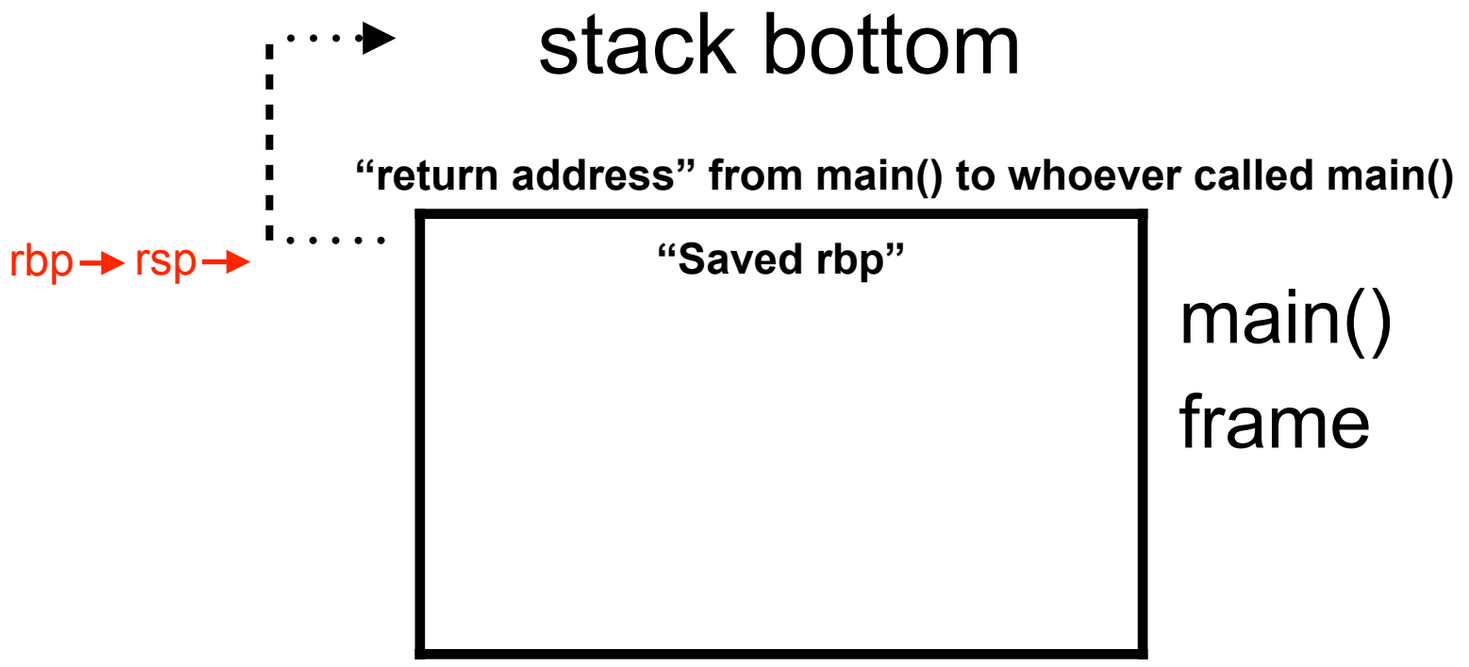


64-bit Maximum Stack Diagram - SysV



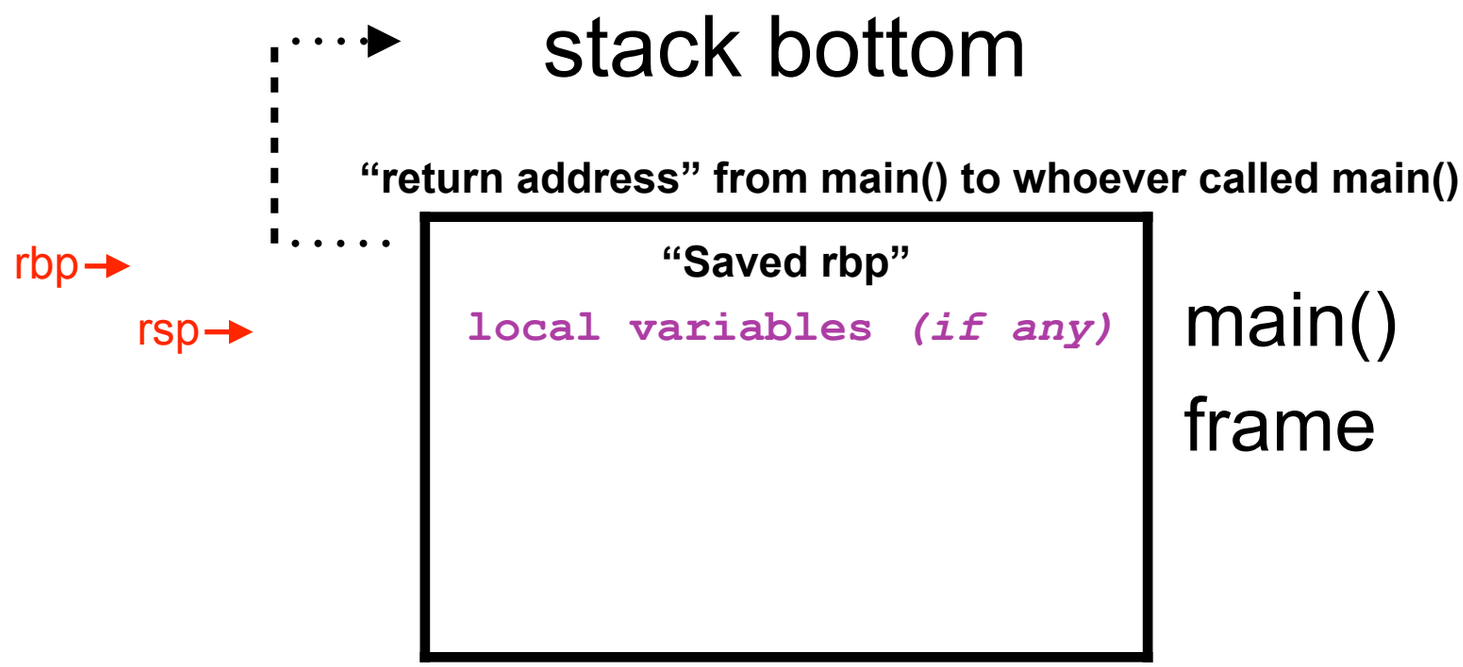
stack top

64-bit Maximum Stack Diagram - SysV



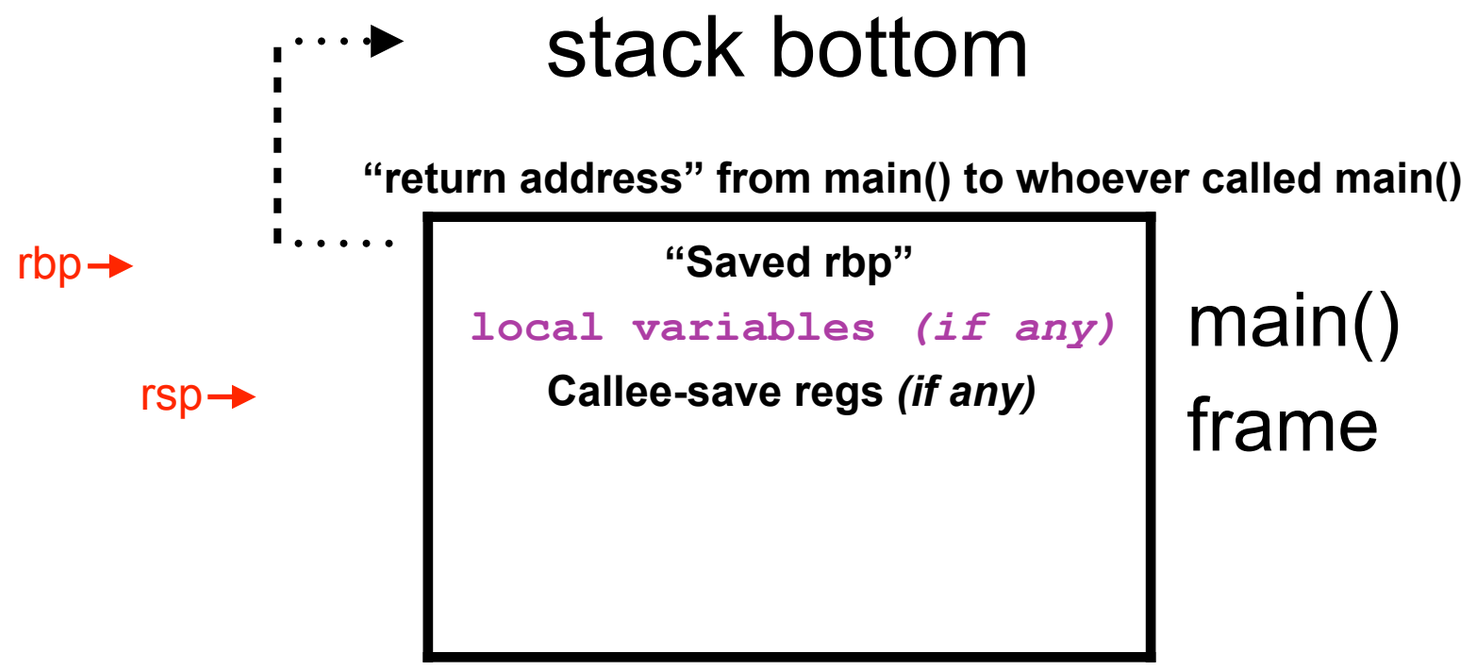
stack top

64-bit Maximum Stack Diagram - SysV



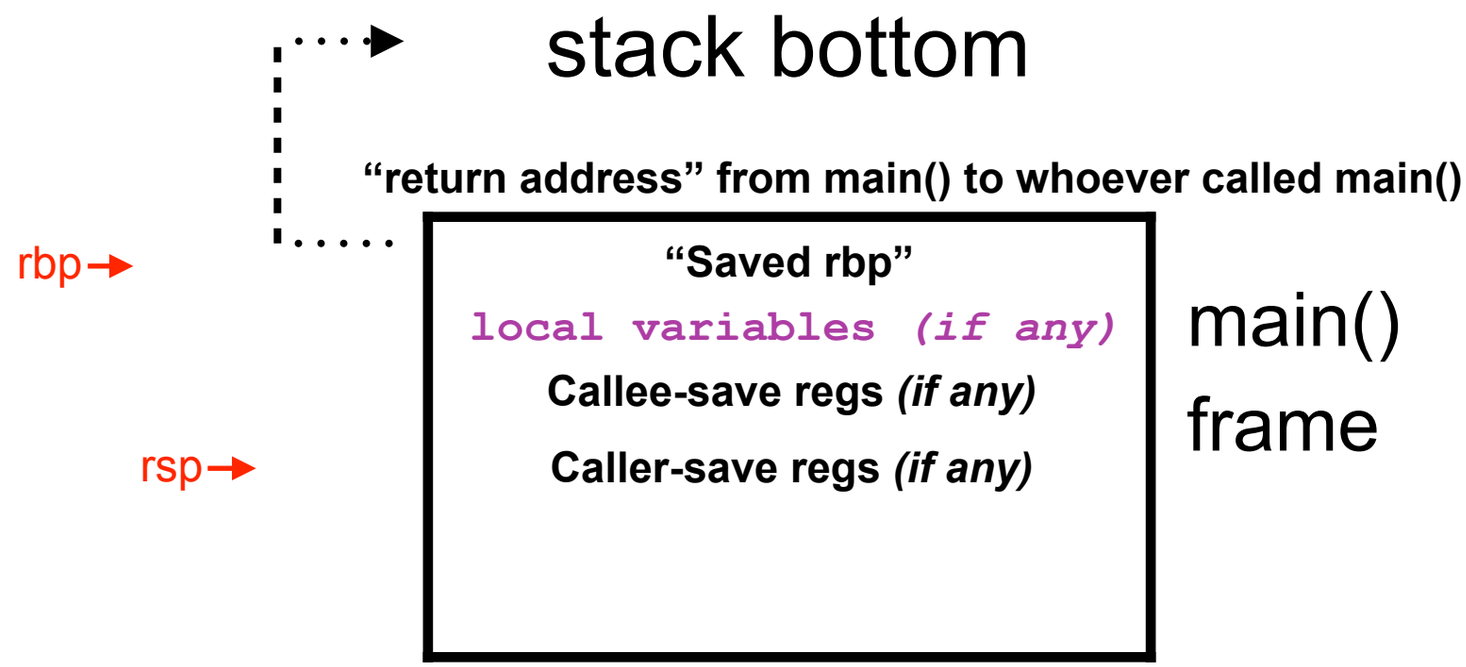
stack top

64-bit Maximum Stack Diagram - SysV



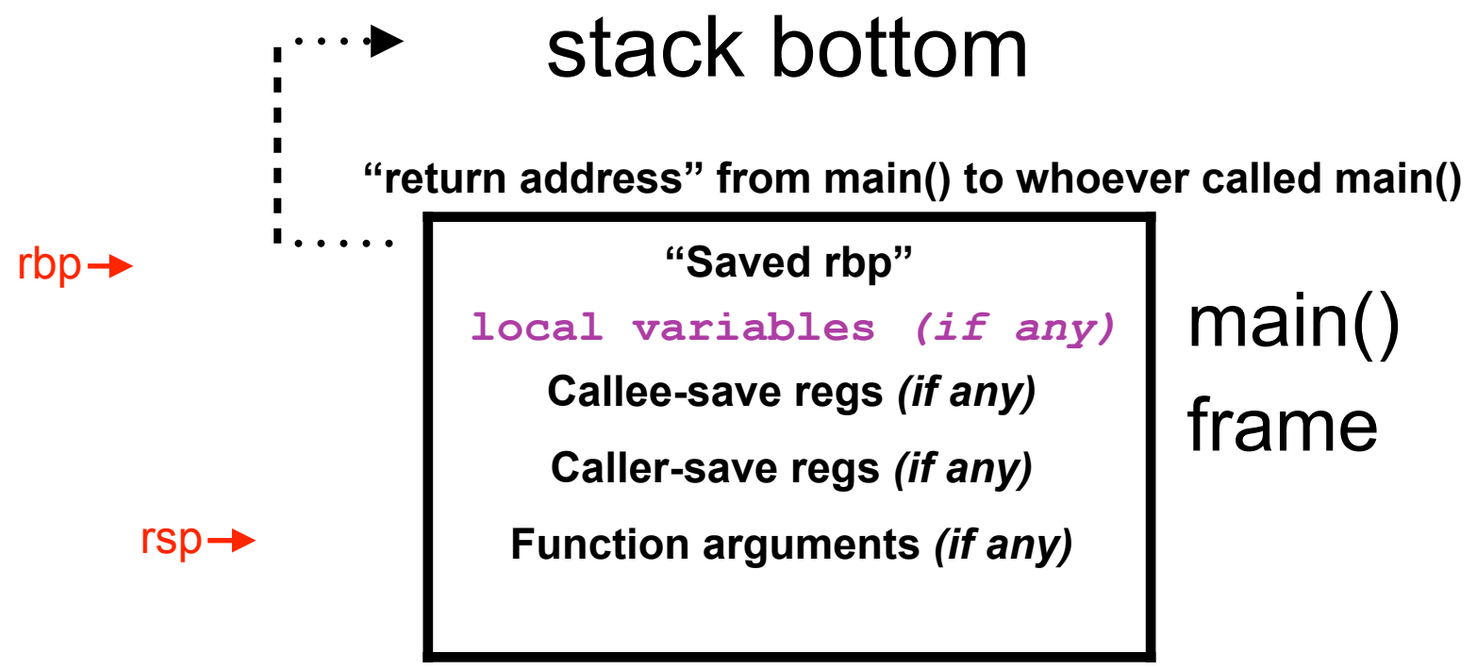
stack top

64-bit Maximum Stack Diagram - SysV



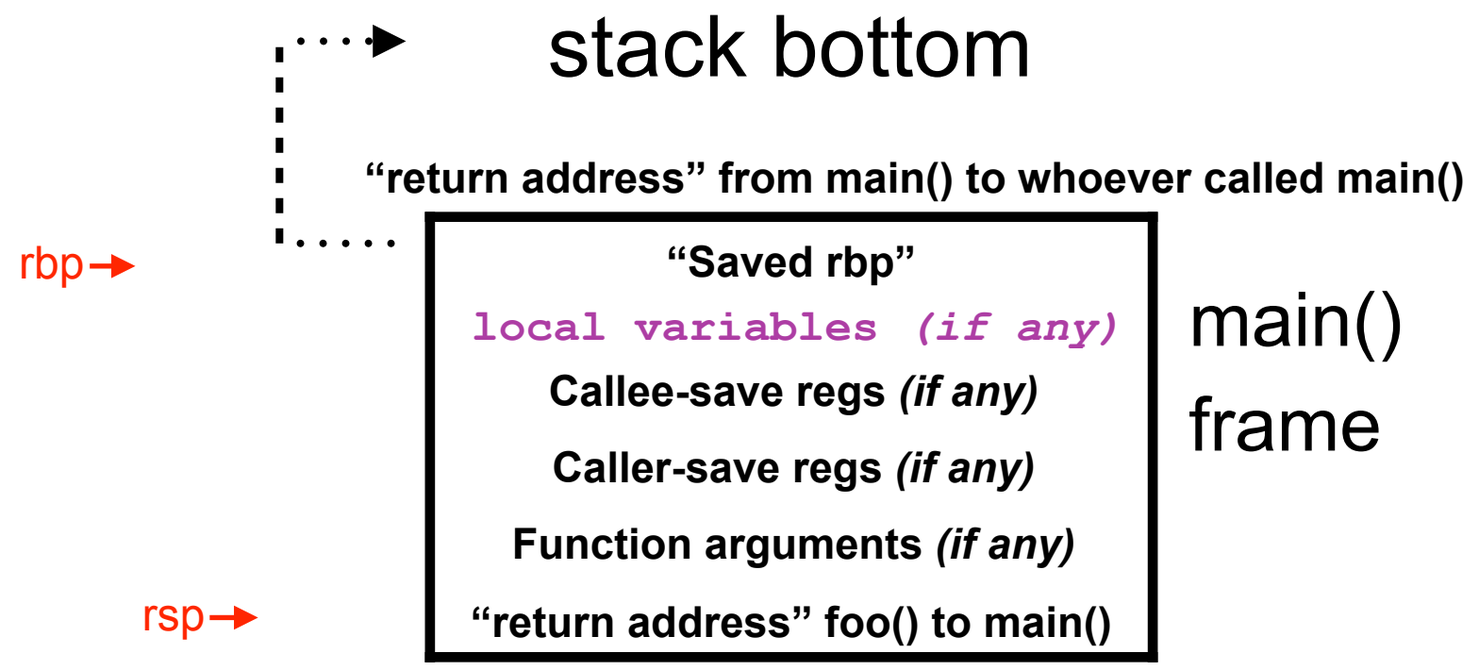
stack top

64-bit Maximum Stack Diagram - SysV



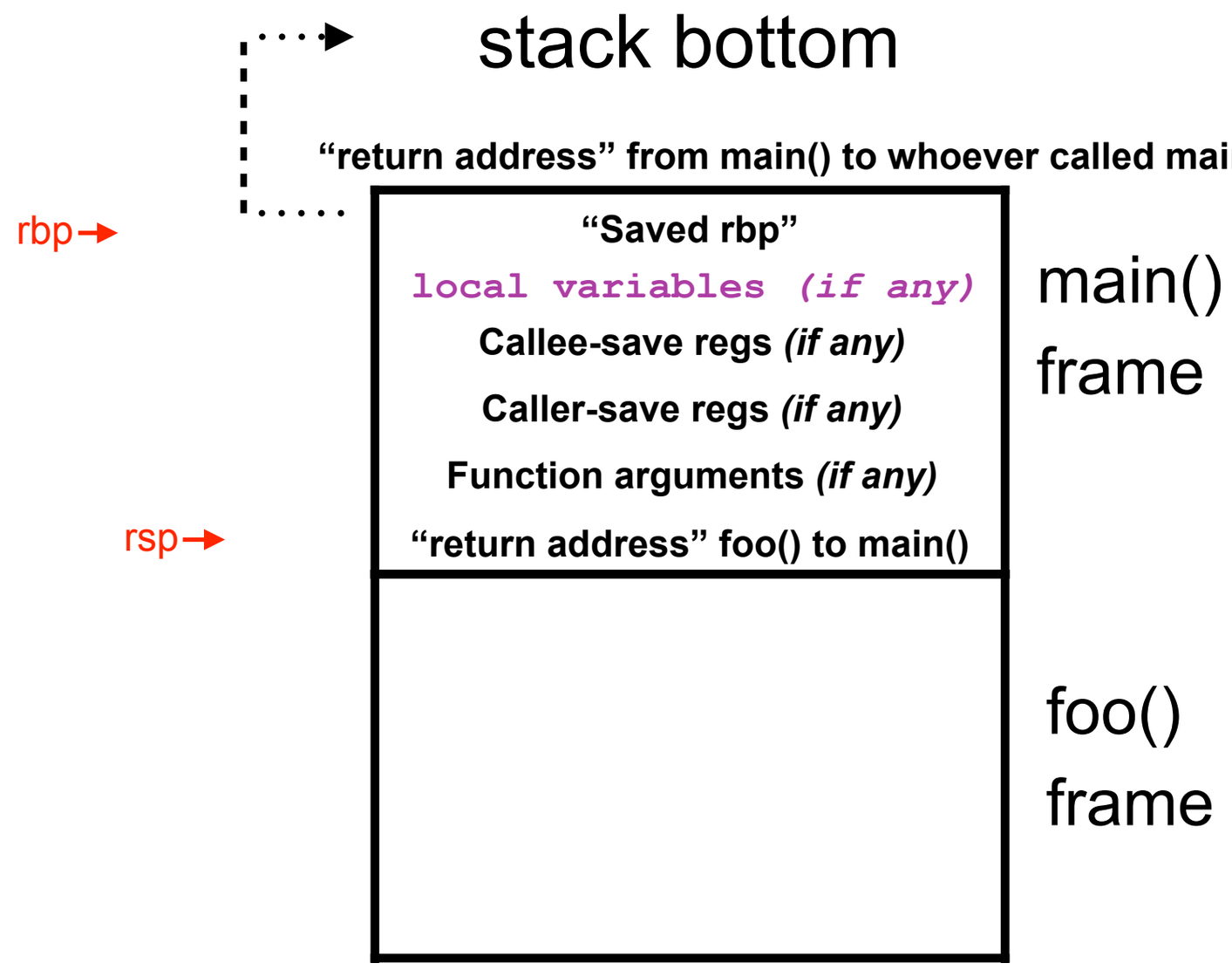
stack top

64-bit Maximum Stack Diagram - SysV



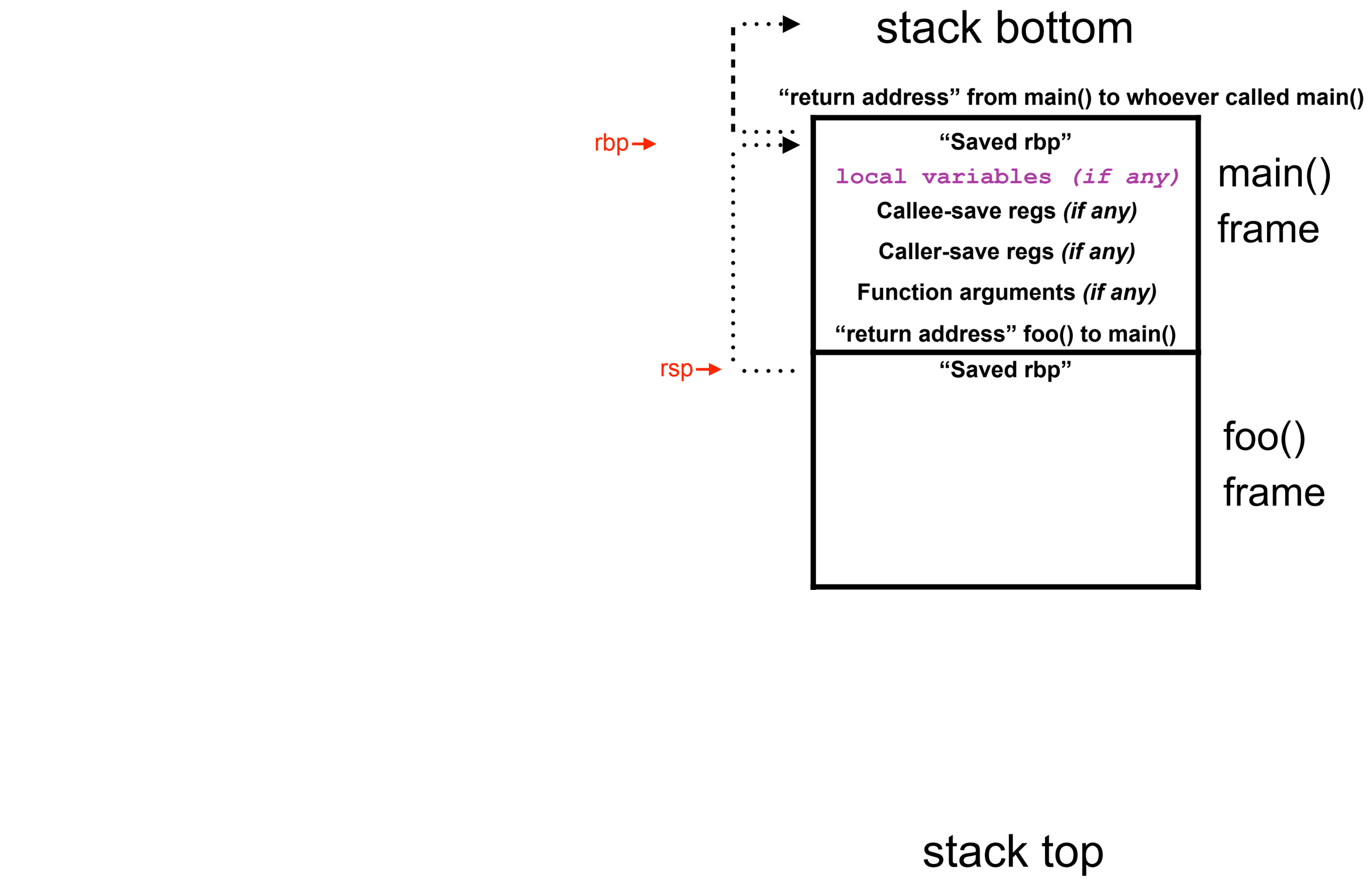
stack top

64-bit Maximum Stack Diagram - SysV

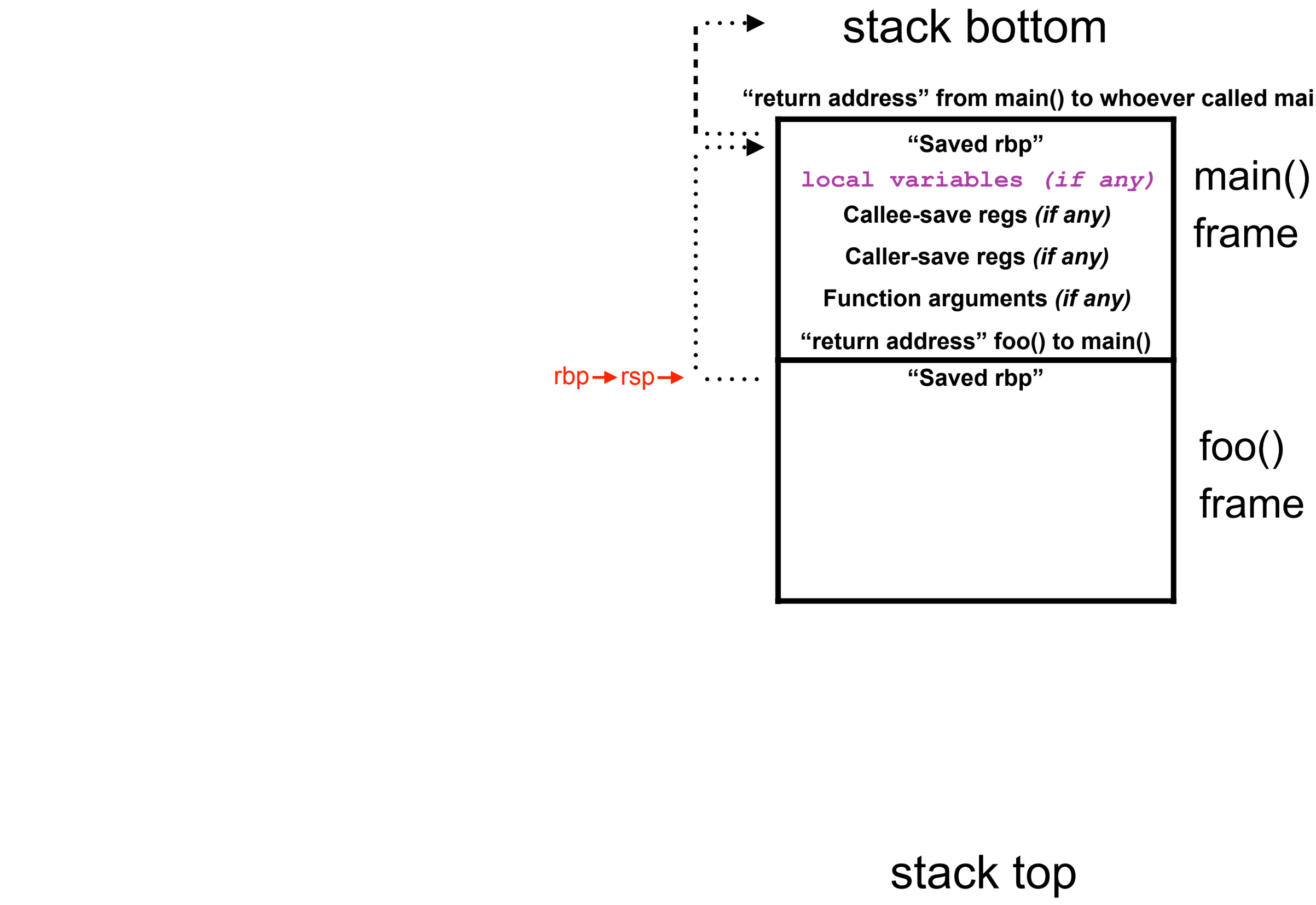


stack top

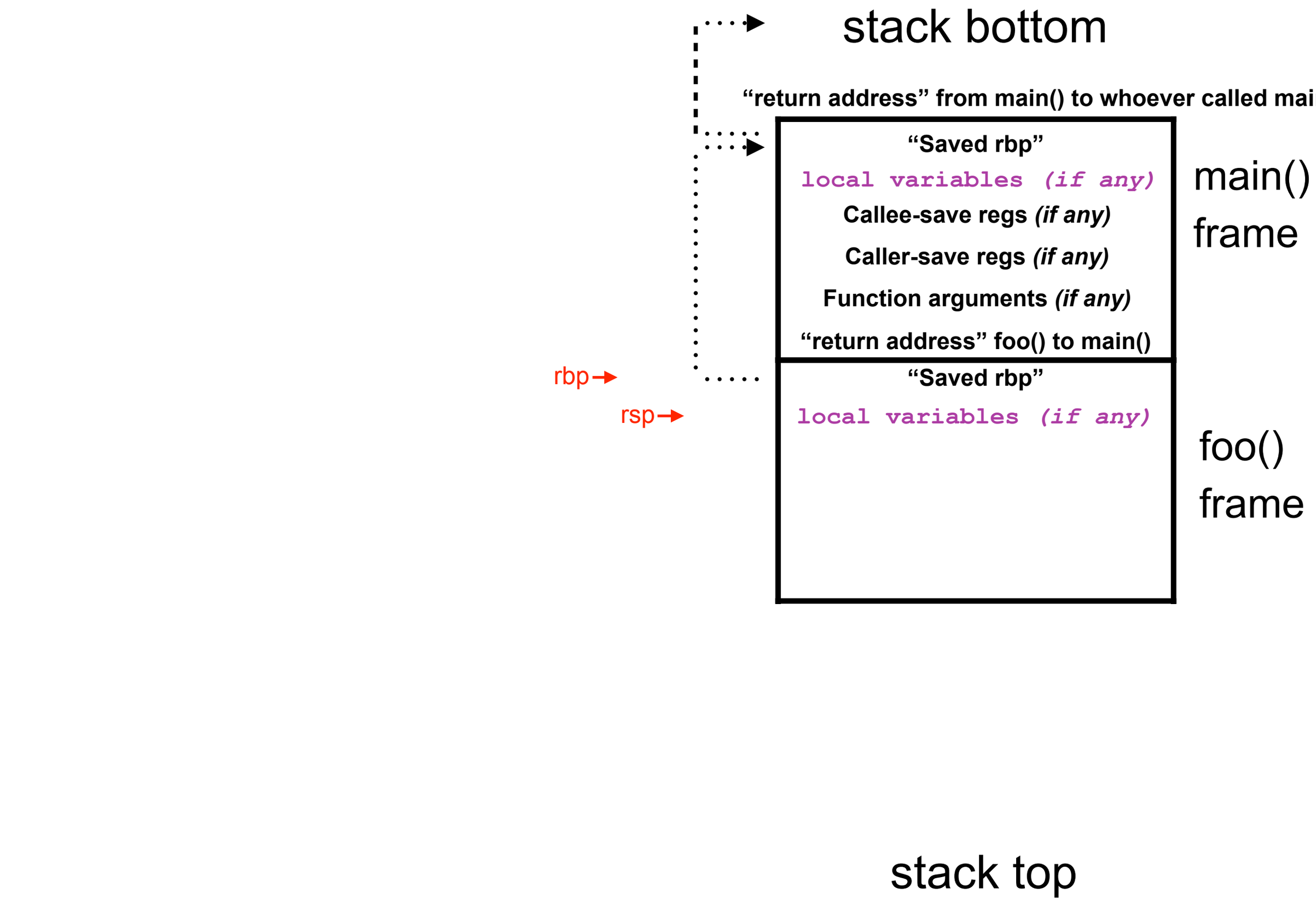
64-bit Maximum Stack Diagram - SysV



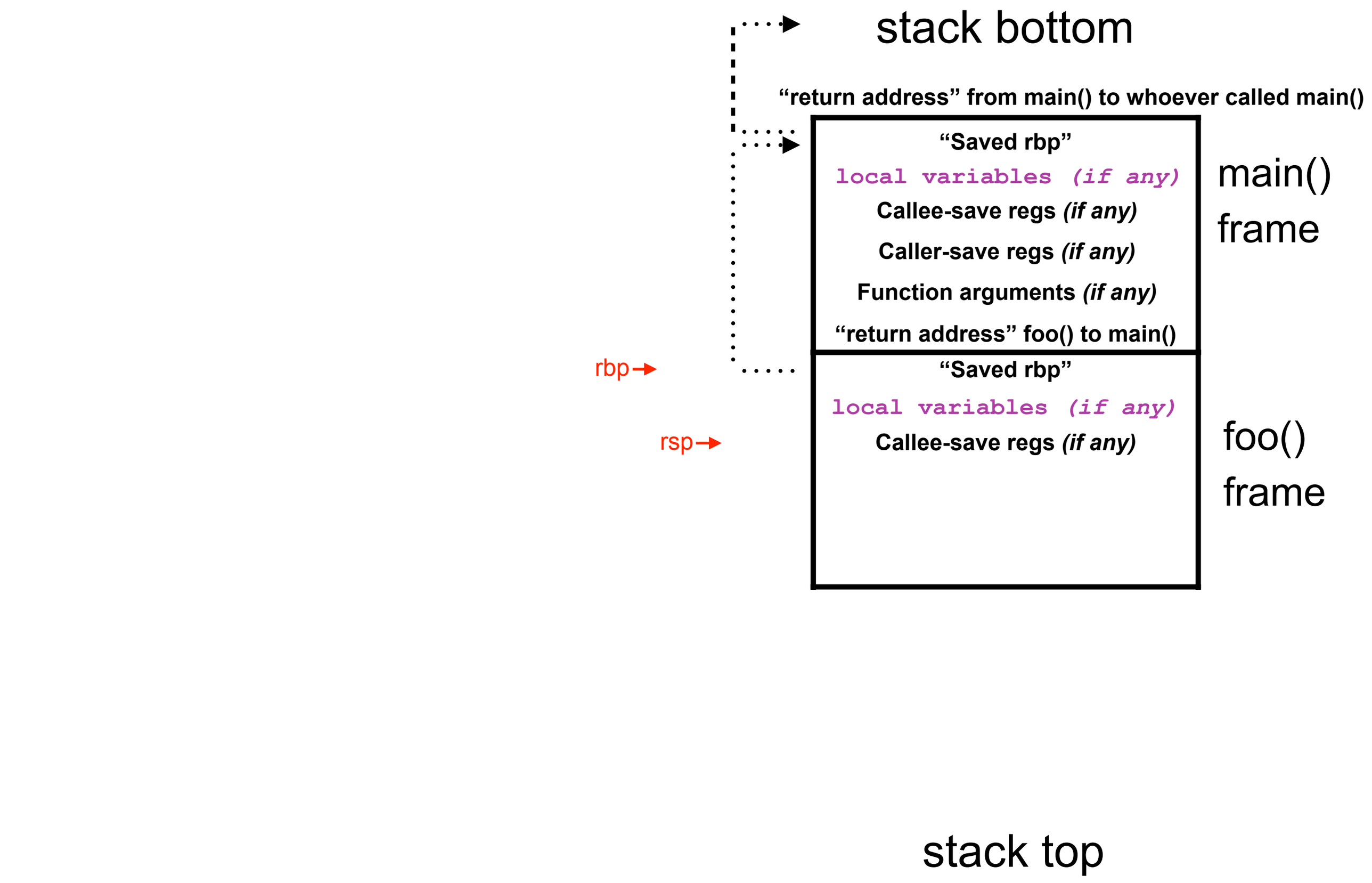
64-bit Maximum Stack Diagram - SysV



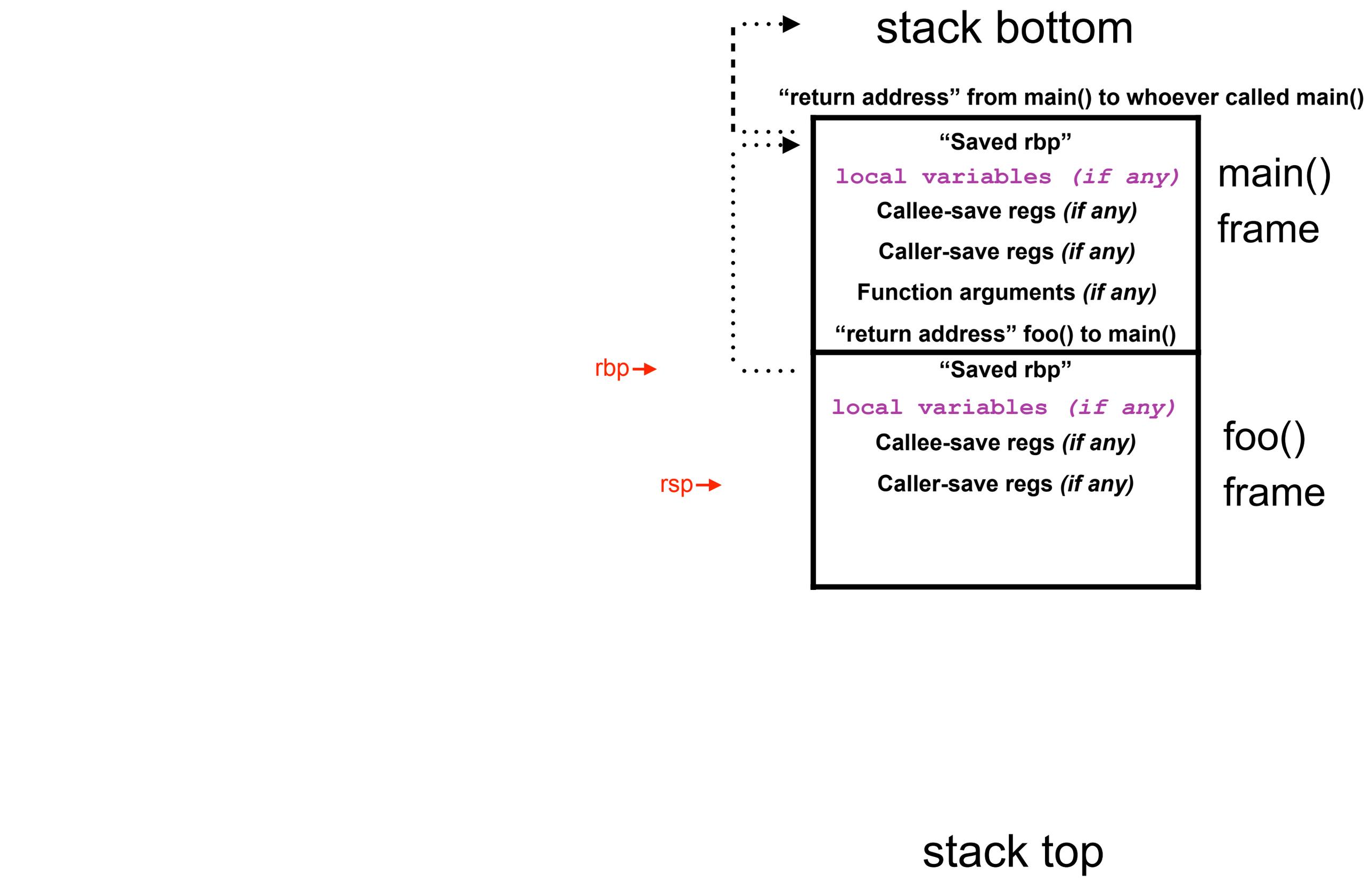
64-bit Maximum Stack Diagram - SysV



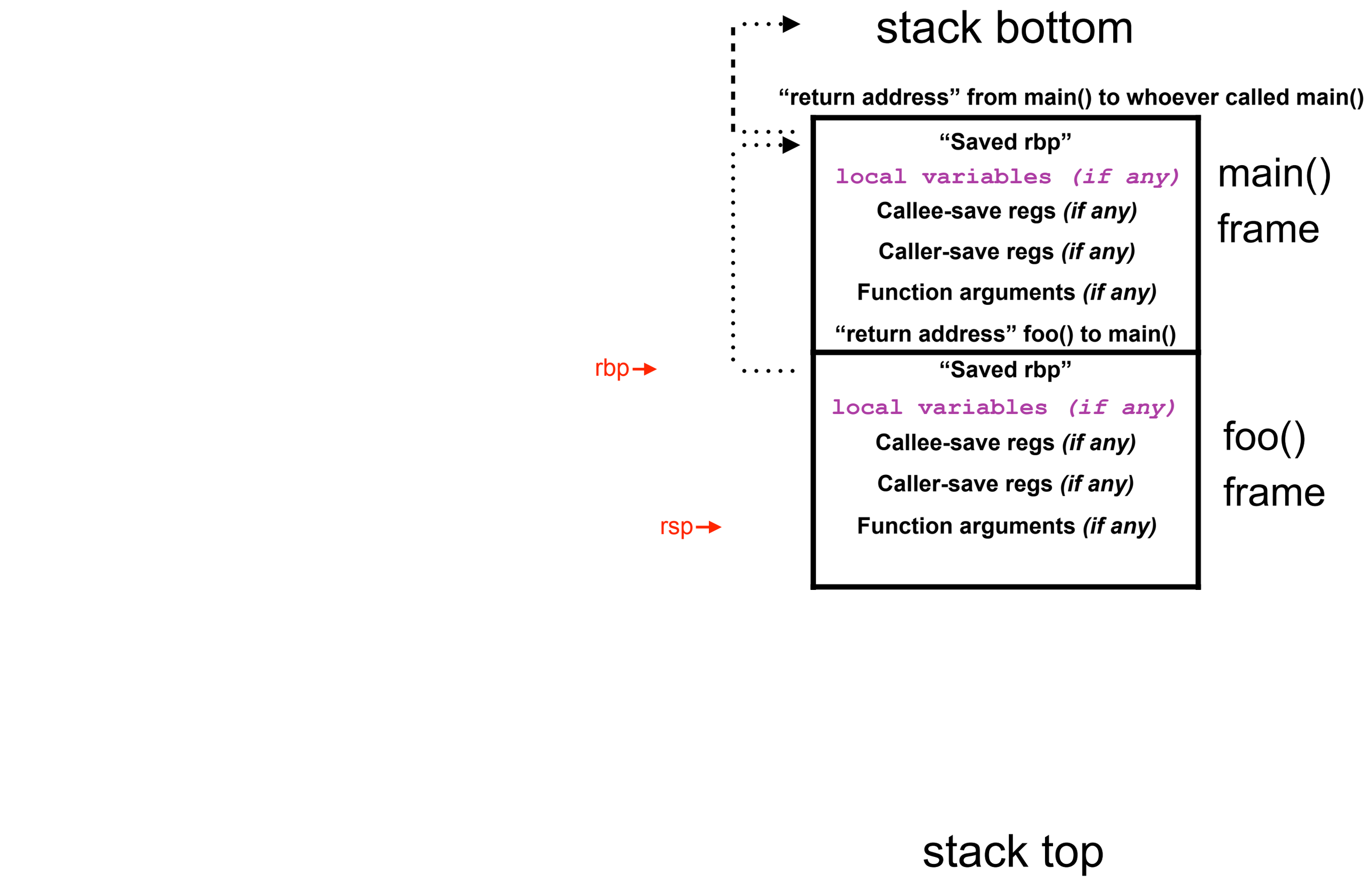
64-bit Maximum Stack Diagram - SysV



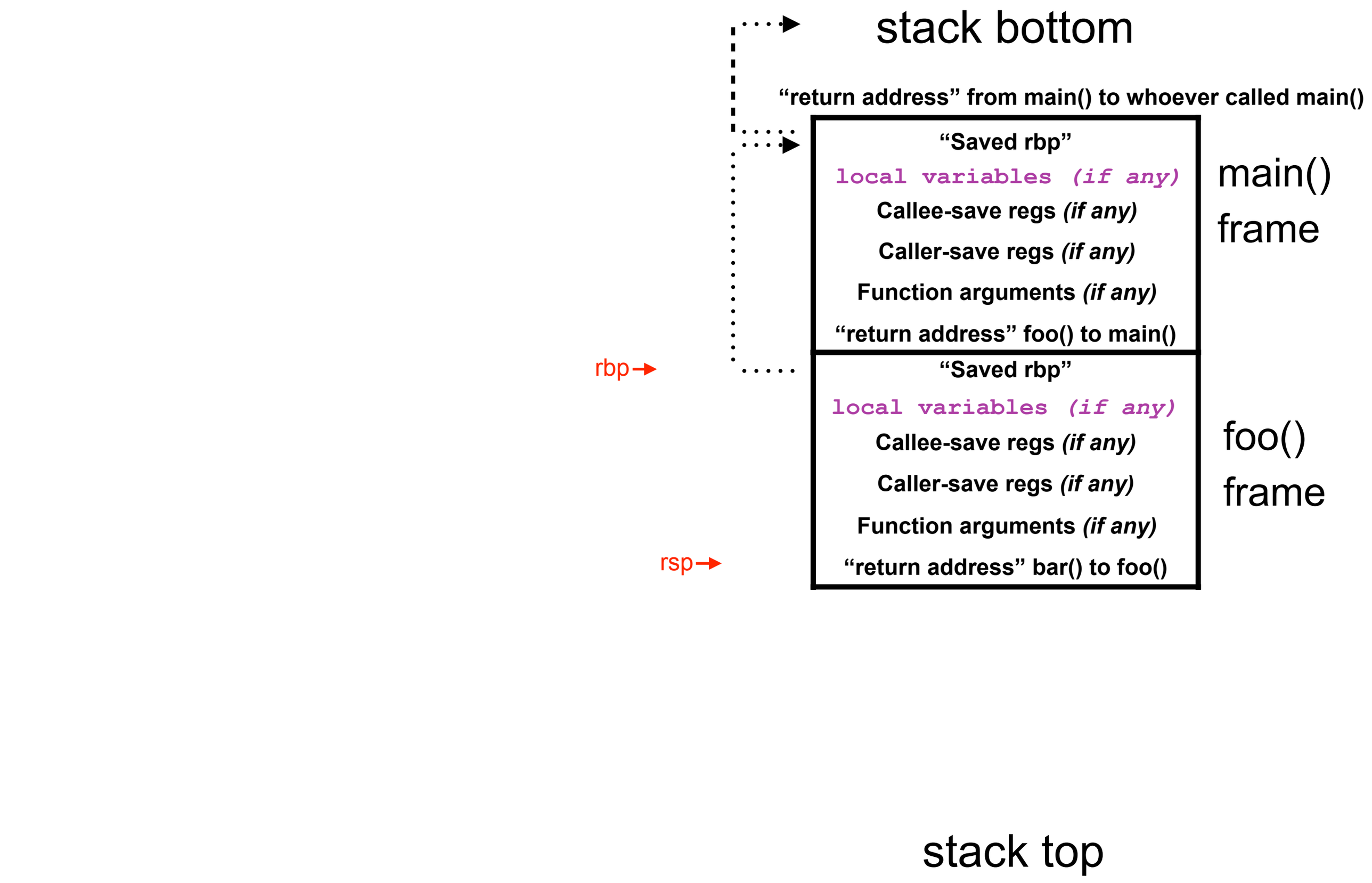
64-bit Maximum Stack Diagram - SysV



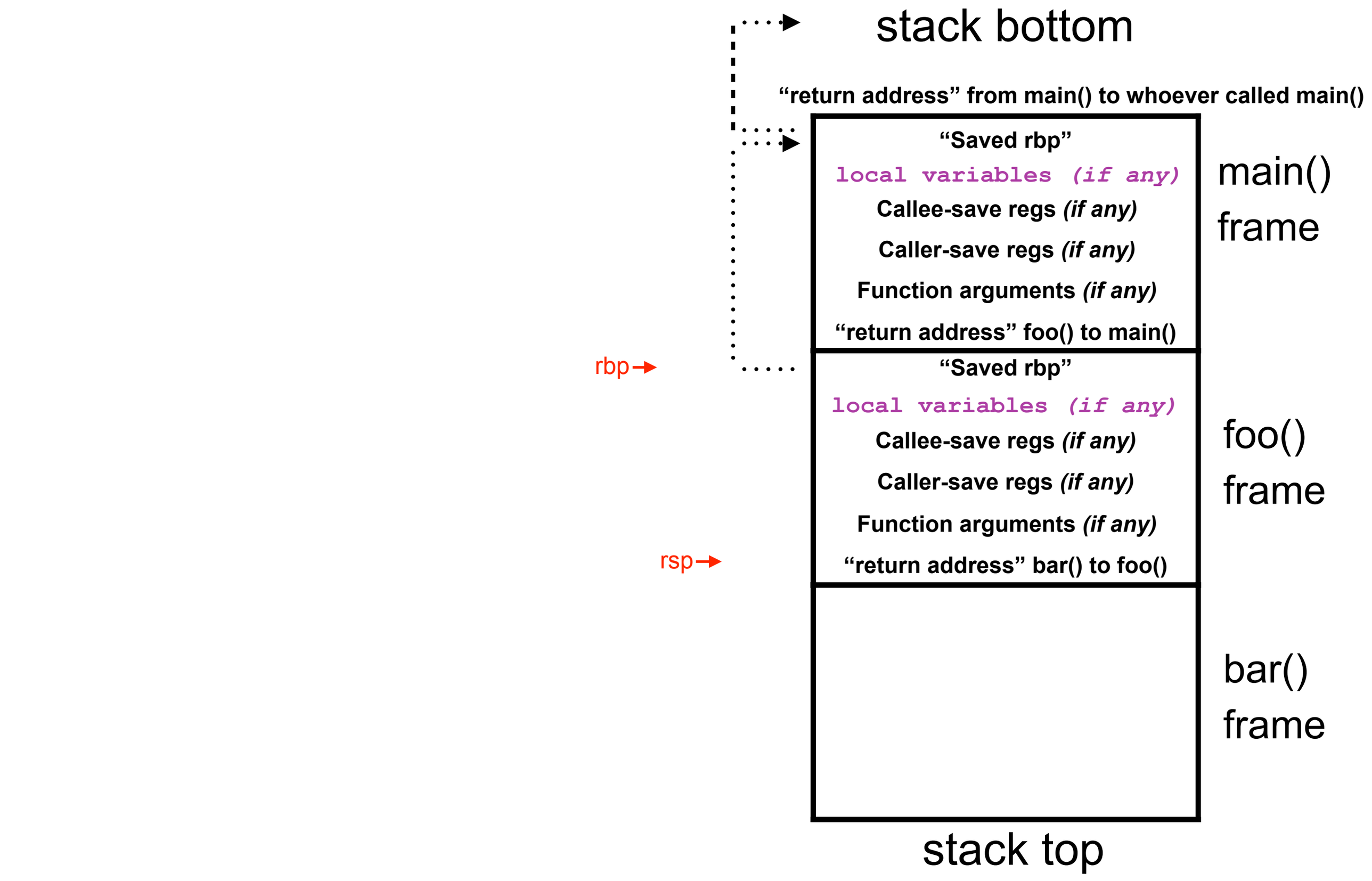
64-bit Maximum Stack Diagram - SysV



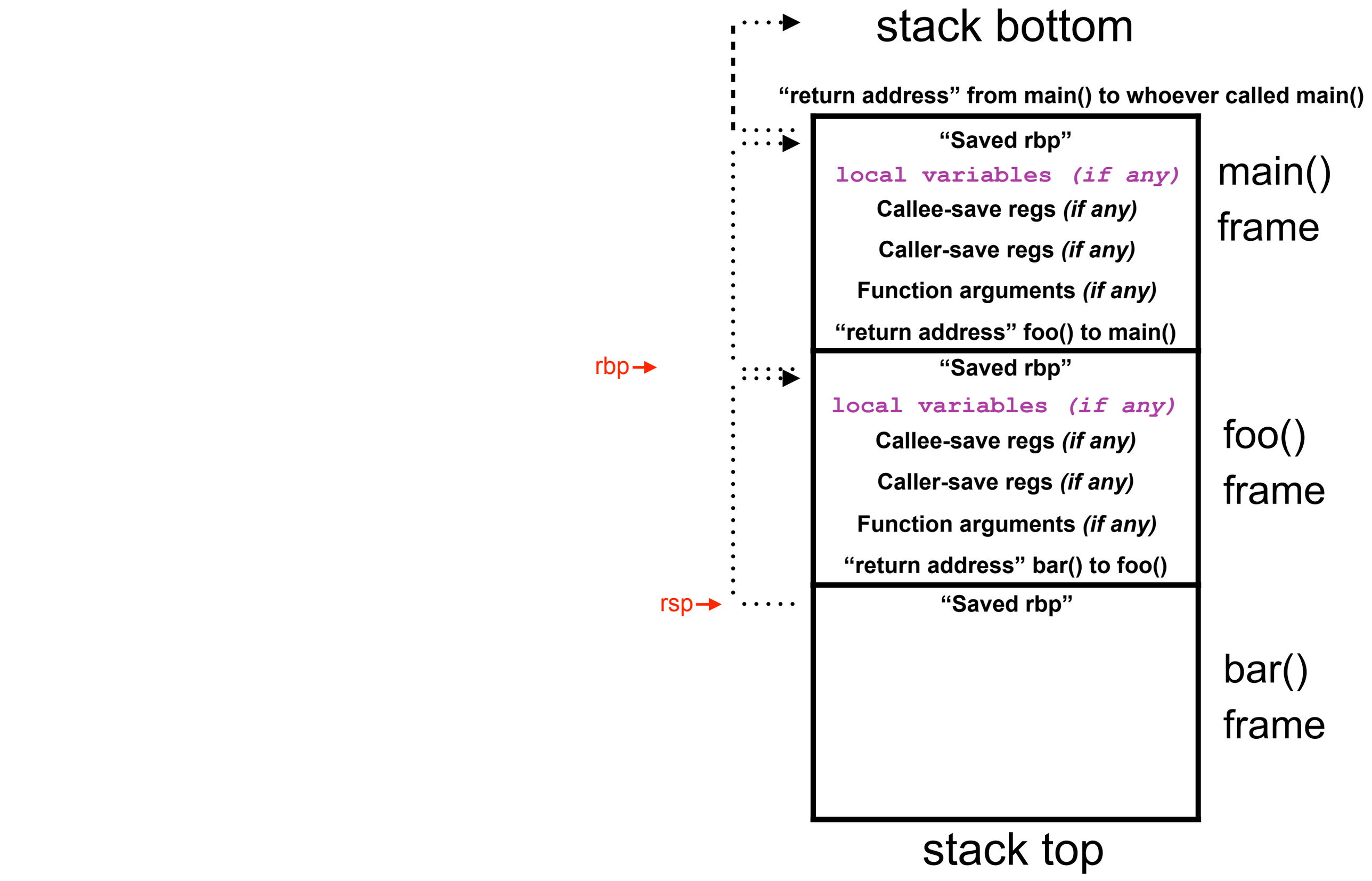
64-bit Maximum Stack Diagram - SysV



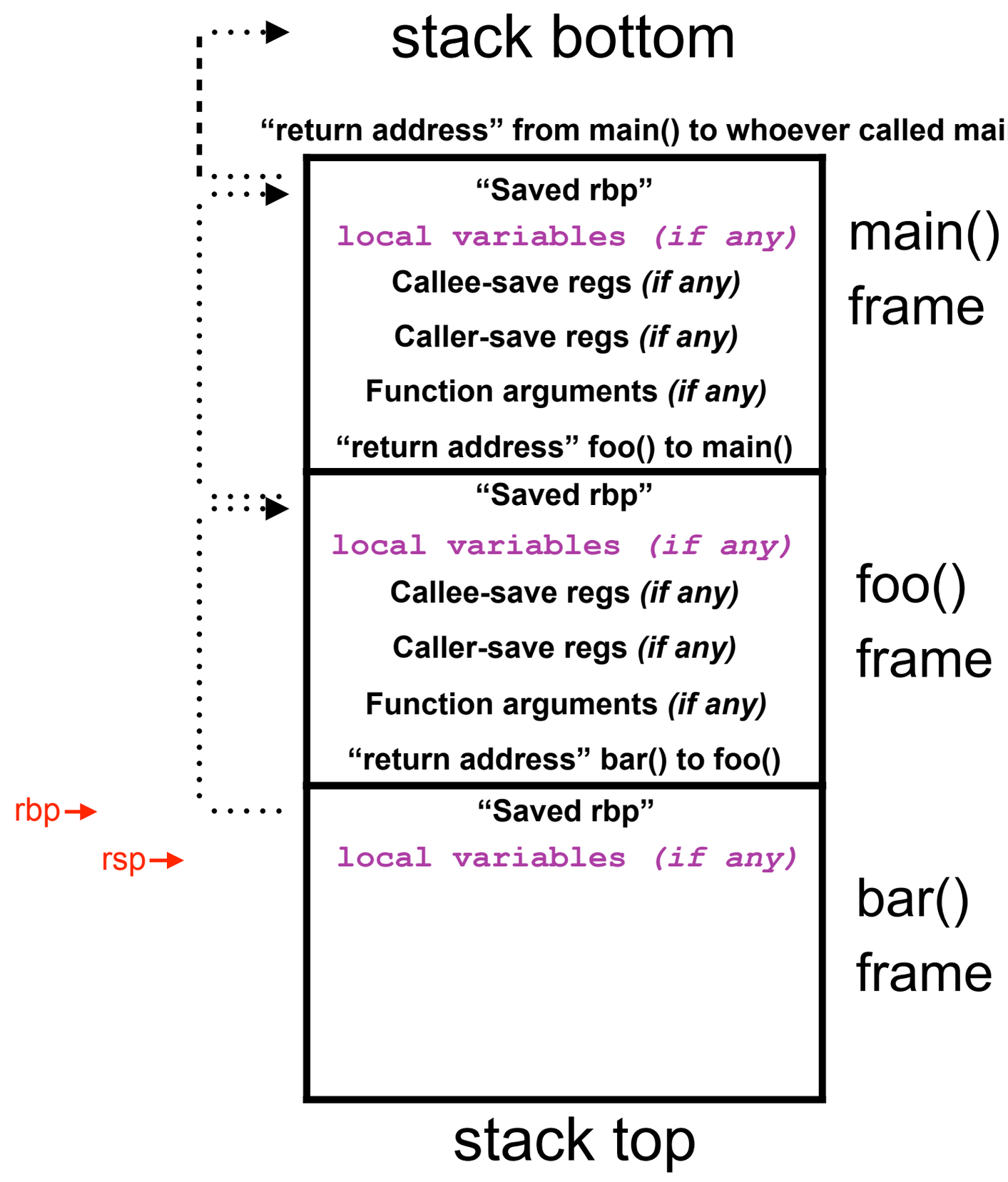
64-bit Maximum Stack Diagram - SysV



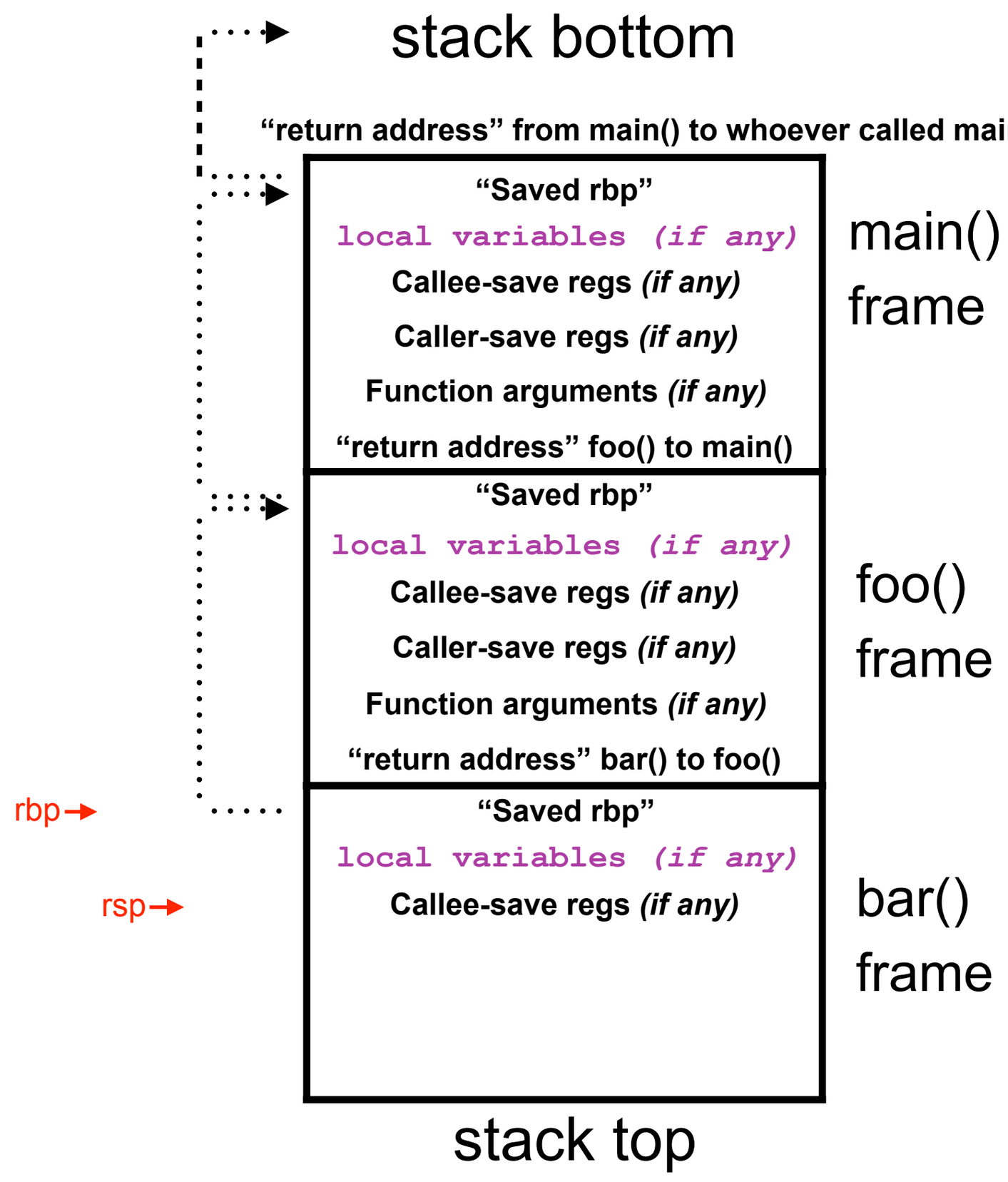
64-bit Maximum Stack Diagram - SysV



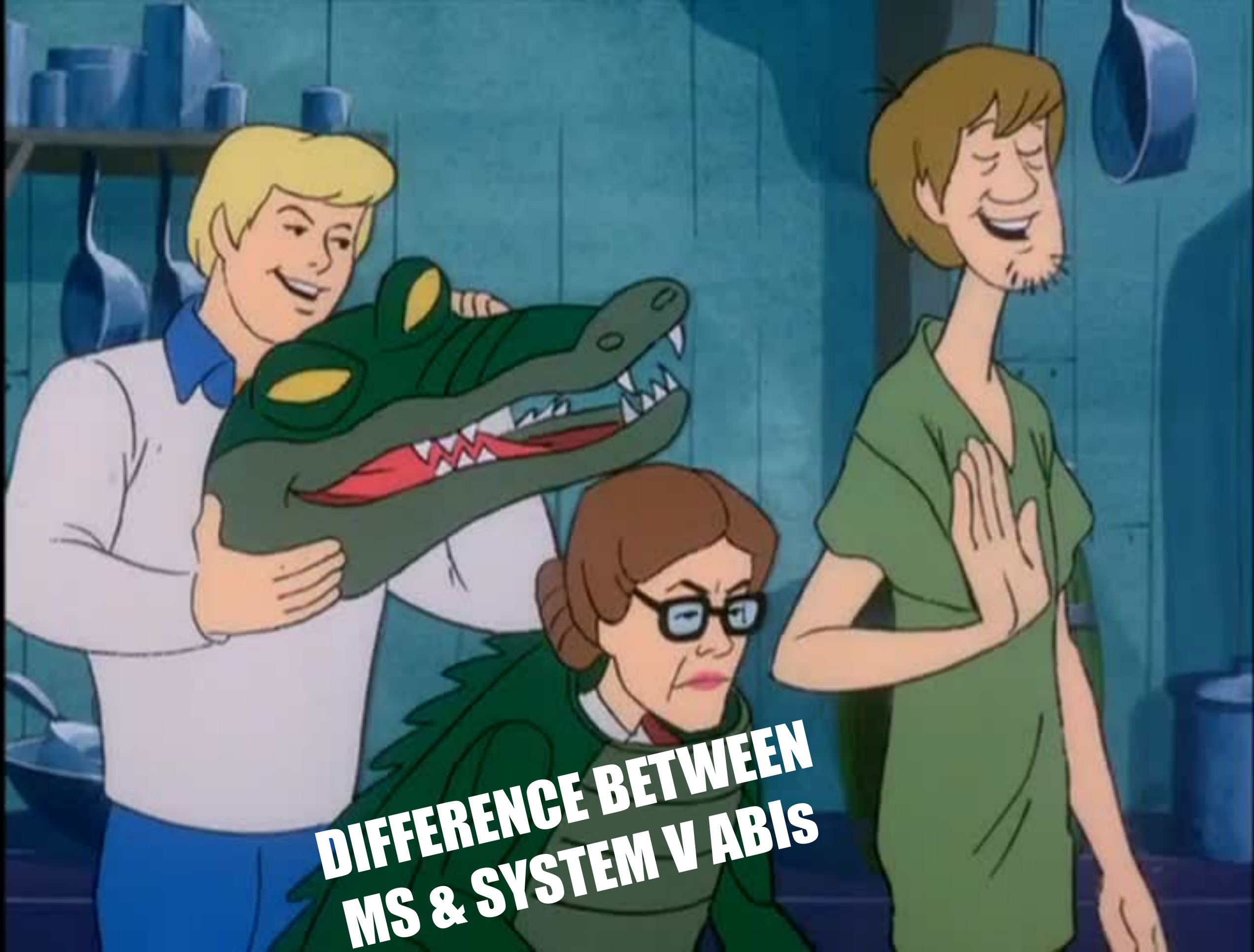
64-bit Maximum Stack Diagram - SysV



64-bit Maximum Stack Diagram - SysV




PRESENCE OF PUSH/POP
IN SYSTEM V CODE
ABSENSE IN MS CODE


DIFFERENCE BETWEEN
MS & SYSTEM V ABIs

Mystery Listery

- Why do the GCC/Clang HelloWorlds have balanced Push/Pop instructions but Visual Studio doesn't?
- What's up with the sub/add 0x28 in main()?
- Why is VS over-allocating space for a single local variable?

SOLVED!

SOLVED!

Mystery Listery

- Why do the GCC/Clang HelloWorlds have balanced Push/Pop instructions but Visual Studio doesn't?
- What's up with the sub/add 0x28 in main()?
- Why is VS over-allocating space for a single local variable?

SOLVED!

SOLVED!

SOLVED!

